

Week3_Sun_800_Professional Software Engineering_Lecture_Note

Professional Software Engineering 课堂笔记与翻译

Professional Software Engineering - Week 3 Lecture Notes

Course Administration and Logistics

LMS ACCESS AND TECHNICAL ISSUES

Mohammad asks about LMS access for week 3 content.

 **Student:** What's that, sir?  **Mohammad:** Do you have access to the LMS?

Speaker 2 confirms access.

Mohammad provides guidance for students without LMS access:

- If you joined recently (just this week), ask your friend about the LMS
- If they cannot help, I can assist you at the end of the class

Mohammad identifies four students without LMS access:

- Please text me on Teams to say "hi"
- I will add you and send a screenshot of the current access

Wi-Fi AND INTERNET CONNECTIVITY

Mohammad addresses education Wi-Fi issues:

- Did you talk to **Arun** (your team lead)?
- If you leave it, no one will solve it — you need to follow up

- I cannot resolve internet issues at my level
- You need to check with the team lead
- Maybe you need to open a ticket with IT to get support

 **Speaker 3:** Thank you.

Mohammad asks for any questions.

 **Speaker 2:** Did you start working on your assignment?  **Mohammad:** This course?

 **Speaker 2:** Yeah, this course.  **Speaker 2:** You, sir.  **Speaker 2:** I was thinking of studying.

Mohammad acknowledges.

Core Academic Content: Data Access and Databases

WEEK 3 OVERVIEW

In week 3, we will discuss:

- Accessing data files
- Different databases you need to connect with your projects

Mohammad explains the week 3 materials:

- Two PowerPoint presentations in Blackboard (A and B)
 - **Presentation A:** Working with different files (e.g., text files)
 - **Presentation B:** Related to databases — different databases

SQLITE3: BUILT-IN RELATIONAL DATABASE

One of the databases we will discuss today is **SQLite3**.

Key characteristics of SQLite3:

- It is a **built-in database**

- No setup required for connection with your application
- Installation is similar to other libraries or packages
- It is a **relational database**

RELATIONAL VS. NON-RELATIONAL DATABASES

Mohammad introduces the fundamental distinction:

Relational Databases:

- Examples: MySQL, SQLite3, Oracle, Microsoft SQL Server
- Used for structured, tabular data
- Example scenario: Developing a project for our college to handle student information, results, etc.

Non-Relational Databases:

- Used for unstructured data
- Example: Video streaming applications (like YouTube)
- Video data is non-structured

Important Distinction: Do not confuse *relational vs. non-relational databases* with *structured vs. unstructured data* — these are two different concepts.

STRUCTURED VS. UNSTRUCTURED DATA

Mohammad clarifies the data structure concepts:

Data Type	Characteristics	Examples
Structured Data	Tabular format	Student records, results
Unstructured Data	No predefined format	YouTube video content

Future Course Connection: Next semester, you will focus on the *Data Analysis* course, where you will work with both structured and unstructured data.

DATABASE EXAMPLES

Mohammad lists common database systems:

- MySQL
 - SQLite3
 - Oracle
 - Microsoft SQL Server
-

中文全文翻译

专业软件工程 - 第三周课程笔记

课程管理与后勤事务

LMS访问与技术问题

Mohammad询问学生是否能够访问第三周内容的LMS。

 学生：先生，那是什么？  **Mohammad**：你能访问LMS吗？

Speaker 2确认可以访问。

Mohammad为无法访问LMS的学生提供指导：

- 如果你是最近（就在本周）才加入的，请向你的朋友询问LMS
- 如果他们无法帮忙，我可以在课程结束时协助你

Mohammad确认有四名学生无法访问LMS：

- 请在Teams上给我发消息说"hi"
- 我会添加你并发送当前访问权限的截图

Wi-Fi与网络连接

Mohammad处理教育Wi-Fi问题：

- 你和**Arun**（你的团队负责人）谈过了吗？

- 如果你放任不管，没人会解决——你需要跟进
- 我无法解决网络问题，因为我没有那个权限
- 你需要与团队负责人核实
- 也许你需要向IT部门提交工单以获得支持

 **Speaker 3**：谢谢。

Mohammad询问是否有任何问题。

 **Speaker 2**：你开始做作业了吗？ **Mohammad**：这门课？ **Speaker 2**：是的，这门课。 **Speaker 2**：先生，是问你。 **Speaker 2**：我本来想开始学习的。

Mohammad表示了解。

核心学术内容：数据访问与数据库

第三周概述

在第三周，我们将讨论：

- 访问数据文件
- 你需要连接到项目的不同数据库

Mohammad解释第三周的材料：

- Blackboard上有两个PowerPoint演示文稿（A和B）
 - **演示文稿A**：处理不同类型的文件（例如，文本文件）
 - **演示文稿B**：与数据库相关——不同的数据库

SQLite3：内置关系型数据库

我们今天要讨论的数据库之一是**SQLite3**。

SQLite3的主要特点：

- 它是一个**内置数据库**
- 无需为与应用程序的连接进行任何设置
- 安装方式与其他库或包类似

- 它是一个关系型数据库

关系型数据库与非关系型数据库

Mohammad介绍了基本区别：

关系型数据库：

- 示例：MySQL、SQLite3、Oracle、Microsoft SQL Server
- 用于结构化、表格化数据
- 示例场景：为我们的学院开发一个项目，用于处理学生信息、成绩等

非关系型数据库：

- 用于非结构化数据
- 示例：视频流应用（如YouTube）
- 视频数据是非结构化的

重要区分：不要混淆关系型与非关系型数据库与结构化与非结构化数据——这是两个不同的概念。

结构化数据与非结构化数据

Mohammad澄清数据结构概念：

数据类型	特点	示例
结构化数据	表格格式	学生记录、成绩
非结构化数据	无预定义格式	YouTube视频内容

未来课程关联：下学期，你将专注于数据分析课程，届时你将处理结构化和非结构化数据。

数据库示例

Mohammad列出常见的数据库系统：

- MySQL

- SQLite3
- Oracle
- Microsoft SQL Server

Professional Software Engineering - Level 9

Master of Software Engineering

Database Design Fundamentals

STRUCTURED VS. UNSTRUCTURED DATA

Mohammad explains that databases store different data structures. The data can be classified as:

- **Structured data:** Tabular data with predefined format (as previously discussed)
- **Unstructured data:** Data without predefined format (e.g., YouTube video content)

DATABASE SYSTEMS OVERVIEW

Mohammad lists common database systems:

- MySQL
- SQLite3
- Oracle
- Microsoft SQL Server

These databases can handle both structured and unstructured data. The choice depends on:

- Project size
- Data volume
- Performance requirements

Key Point: Some databases are designed for small to mid-level projects, while others can handle *really large data* (big data) with **no latency** - meaning when you send a query, you get a response quickly.

SQLITE3 AS A STARTING POINT

Mohammad recommends **SQLite3** as one of the easiest ways to learn and start integrating databases with Python.

Why SQLite3? It is a *built-in database* in Python.

THE IMPORTANCE OF DATABASE DESIGN

Mohammad emphasizes that before coding and developing the database, we need to get familiar with database design.



Student: Can we develop any database without designing?



Mohammad: No. No. We're going to build the house without any design. For example, we don't care about civil engineering, we don't care about architecture, why we need these things. We're going to start building and then we will end up with a house with three levels, but it will collapse soon.

Mohammad uses the house-building analogy to explain:

- **Building** a database is easy
- **Maintaining** a database is the difficult part
- In the future, you'll want to add more features - how will you handle that?

Example: If you need to add one more table, it's like adding a bedroom to a house. Some houses are three-bedroom houses, but after a couple of years, the family grows and they need one more bedroom. If you just add it at the back of the house, it doesn't look nice and it's not strong enough.

KEY TAKEAWAY

We need to care about the **detail of database design** at the beginning, then build it properly to be able to use the database for a long time, and it should be *efficient*.

ER DIAGRAMS AND RELATIONSHIPS

Mohammad introduces the concept of **Entity Relationship Diagrams (ER or EER - Extended Entity Relationship)**.

The learning path includes:

1. How to design the ER relationship
2. How to map the ER relationship diagram with table generation
3. Understanding relationships:
 - **One-to-one**
 - **One-to-many**
 - **Many-to-many**
4. In which cases to use each relationship type

EXAMPLE-BASED LEARNING

Mohammad proposes starting with an example to explain the concept.

 **Mohammad:** Who worked before with designing databases? Please raise your hand.

Students raise hands

 **Mohammad:** Okay one, databases, couple of you. So I should be reviewing if you developed and designed before. If you are new...

中文全文翻译

专业软件工程 - 软件工程硕士9级

数据库设计基础

结构化数据与非结构化数据

Mohammad 解释说，数据库存储不同的数据结构。数据可以分为：

- **结构化数据：**具有预定义格式的表格数据（如前所述）
- **非结构化数据：**没有预定义格式的数据（例如，YouTube视频内容）

数据库系统概述

Mohammad 列出了常见的数据库系统：

- MySQL
- SQLite3
- Oracle
- Microsoft SQL Server

这些数据库可以处理结构化和非结构化数据。选择取决于：

- 项目规模
- 数据量
- 性能要求

关键点：有些数据库设计用于处理中小型项目，而另一些可以处理*非常大的数据*（大数据），并且*没有延迟*——这意味着当你发送查询时，你会快速得到响应。

SQLite3 作为起点

Mohammad 推荐 **SQLite3** 作为学习和开始将数据库与Python集成的最简单方法之一。

为什么选择 **SQLite3**？它是 Python 中的*内置数据库*。

数据库设计的重要性

Mohammad 强调，在编码和开发数据库之前，我们需要熟悉数据库设计。

 **学生：**我们可以在没有设计的情况下开发任何数据库吗？

 **Mohammad：**不能。不能。我们要在没有设计的情况下建造房子。例如，我们不在乎土木工程，我们不在乎建筑学，为什么我们需要这些东西。我们开始建造，然后最终会得到一栋三层楼的房子，但它很快就会倒塌。

Mohammad 用盖房子的类比来解释：

- 建造数据库很容易
- 维护数据库是困难的部分
- 将来，你会想添加更多功能——你将如何处理？

示例：如果你需要添加一个表，就像给房子增加一个卧室。有些房子是三居室的，但几年后，家庭人口增长，他们需要多一个卧室。如果你只是在房子后面加建，它看起来不美观，也不够坚固。

关键点

我们需要在开始时关注数据库设计的细节，然后正确地构建它，以便能够长期使用数据库，并且它应该是高效的。

ER图和关系

Mohammad 介绍了实体关系图（ER 或 EER - 扩展实体关系）的概念。

学习路径包括：

1. 如何设计ER关系
2. 如何将ER关系图映射到表生成
3. 理解关系类型：
 - 一对一
 - 一对多
 - 多对多
4. 在哪些情况下使用每种关系类型

基于示例的学习

Mohammad 建议从一个示例开始来解释这个概念。

 **Mohammad**：谁以前做过数据库设计？请举手。

学生们举手

 **Mohammad**：好的，一位，数据库，你们中有几位。所以如果你们以前开发和设计过，我应该进行复习。如果你是新手...

好的，这是根据您的要求，对提供的字幕文件进行处理后的结果。

1. ZERO INFORMATION LOSS & STRUCTURAL REORGANIZATION (ENGLISH ONLY)

Mohammad suggests starting with an example to explain the concept of relationships (one-to-one, one-to-many, many-to-many).

 **Student:** *[Students raise hands]*  **Mohammad:** Who worked before with designing databases? Please raise your hand. Okay, one, databases, a couple of you. So I should be reviewing if you developed and designed before.

Mohammad advises new students to take extra time to design the ER (Entity-Relationship) diagram for this project. He will explain the concept.

STARTING WITH A SIMPLE EXAMPLE

Mohammad will begin with a simple example for today's class.

 **Mohammad:** We're going to start with a simple example for today. And your team. Week 3. So, I can't wait to see the activity one.

The goal is to design the ER (Entity-Relationship) diagram for our class and students.

UNDERSTANDING THE BASICS: TABLES, ATTRIBUTES, AND RECORDS

Mohammad explains the fundamental components of a database table.

- **Number of Courses:** You can imagine how many courses we have this semester, 3. The exact number (30, 40, 50) is not important now because it represents the number of records in the table.
- **Key Focus:** The most important thing is how many attributes are available.
- **Basic Information:** If you have one table, it contains the basic information you need to know.
- **Accessing Materials:** If you cannot see the slides from the back, you can open this file from Week 3 on your Blackboard.

Mohammad defines key database terms:

- **Attribute:** When we talk about an attribute, it means how many attributes we have. It depends on the number of columns in your table. If your table is designed including an attribute, it means you have ten features. For an attribute in the database, it means ten columns.
- **Row/Record:** A row is a record for your table.
- **Table Name:** Each table should have one unique name, which we call its identity.
- **General Definition:** This is a general definition for each table.

INTRODUCTION TO SQL AND DATABASE INTEGRATION

Mohammad introduces the database system they will use.

- **Choice of Database:** There are different databases that we can develop. We will learn one, and the rest are quite similar.
- **Reason for Choice:** We're going to start with the most useful and easiest for the mid-level project to be integrated with Python.
- **Basic Queries:** We need basic queries. If you don't know them, you need to learn the basics like `SELECT`, `INSERT`, and `DROP`. This is the minimum requirement.
- **Future Topics:** I'm not talking about complex queries for now. Later on, for aggregation in data analysis, you should add that section next semester.
- **Learning Resources:** For now, for basic queries, if you don't know them, there is a single link with lots of examples available. Through this link, you can familiarize yourself.
- **Example Query:** If you want to retrieve a couple of records with a specific attribute, you can `SELECT` from your table. We can join tables using different types of joins and many more.
- **Current Goal:** Make it easy and simple for now to be able to develop the first project with the database. We will add more on top of that later.

DEFINING THE ENTITY

Mohammad returns to the core concept of the ER diagram.

 **Mohammad:** What is the entity? We are talking about ER. I'm alright? ER. Entity. Any out for the entity? Activity. I'm going to put it in the parentheses for now. Activity. Can you see the first activity in...

2. FULL CHINESE TRANSLATION (中文全文翻译)

基于示例的学习

Mohammad 建议从一个示例开始来解释关系（一对一、一对多、多对多）的概念。

 学生: [学生们举手]  **Mohammad:** 谁以前做过数据库设计? 请举手。好的, 一位, 数据库, 你们中有几位。所以如果你们以前开发和设计过, 我应该进行复习。

Mohammad 建议新学生多花些时间来为这个项目设计 ER (实体-关系) 图。他会解释这个概念。

从一个简单的例子开始

Mohammad 将从今天课堂的一个简单例子开始。

 **Mohammad**: 我们今天从一个简单的例子开始。还有你们的团队。第三周。所以，我迫不及待地想看到第一个活动。

目标是设计我们班级和学生的 ER（实体-关系）图。

理解基础：表、属性和记录

Mohammad 解释了数据库表的基本组成部分。

- **课程数量**: 你可以想象我们这个学期有多少门课，3门。确切的数字（30、40、50）现在不重要，因为它代表的是表中的记录数。
- **关键焦点**: 最重要的是有多少个属性可用。
- **基本信息**: 如果你有一个表，它包含你需要知道的基本信息。
- **获取材料**: 如果你坐在后面看不到幻灯片，你可以从 Blackboard 上打开第三周的这个文件。

Mohammad 定义了关键的数据库术语：

- **属性**: 当我们谈论属性时，指的是我们有多少个属性。它取决于你表中的列数。如果你的表设计包含一个属性，意味着你有十个特征。对于数据库中的一个属性，它意味着十列。
- **行/记录**: 一行是你的表的一条记录。
- **表名**: 每个表应该有一个唯一的名称，我们称之为它的标识。
- **通用定义**: 这是每个表的通用定义。

SQL 和数据库集成简介

Mohammad 介绍了他们将使用的数据库系统。

- **数据库的选择**: 我们可以开发不同的数据库。我们将学习一种，其他的都非常相似。
- **选择原因**: 我们将从最有用、最容易与 Python 集成的中级项目开始。
- **基本查询**: 我们需要基本查询。如果你不知道，你需要学习像 `SELECT`、`INSERT` 和 `DROP` 这样的基础知识。这是最低要求。
- **未来主题**: 我现在不讨论复杂查询。以后，对于数据分析中的聚合，你应该在下个学期添加那部分内容。

- **学习资源：** 现在，对于基本查询，如果你不知道，有一个链接提供了很多示例。通过这个链接，你可以自学。
- **查询示例：** 如果你想检索具有特定属性的几条记录，你可以从你的表中进行 `SELECT`。我们可以使用不同类型的连接等方式来连接表。
- **当前目标：** 现在让它变得简单易行，以便能够开发第一个带有数据库的项目。我们稍后会在此基础上添加更多内容。

定义实体

Mohammad 回到了 ER 图的核心概念。

 **Mohammad:** 什么是实体？我们在谈论 ER。我没事吧？ER。实体。实体有什么说法吗？活动。我现在先把它放在括号里。活动。你能看到第一个活动在.....

好的，请查收根据您的要求对字幕文件进行的处理。

1. STRUCTURED SUMMARY & LOGICAL HIGHLIGHTING (ENGLISH ONLY)

Defining Entities & The ER Diagram

Mohammad continues the lecture by revisiting the core concepts of an Entity-Relationship (ER) diagram.

 **Mohammad:** What is the entity? We are talking about ER. Entity. Any out for the entity? Activity. I'm going to put it in the parentheses for now. Activity. Can you see the first activity in your team? Okay, your diagram, it means entity relationship diagram. Instead of class, I can make it a little bit... Let's do that.

Mohammad proposes a teaching method: first showing an example ER diagram, explaining it, and then letting students design their own.

 **Mohammad:** I'm going to show you one ER, then I explain and go through that example. Automatically you will understand what does it mean. Then you can design for the first activity.

Student Question on Tools

 **Speaker 3:** What tool can we use to draw?

 **Mohammad:** I will share. There are a couple of free platforms online. You can design your ER. Any of them should be okay. Or if you have the Visio in your system, you can use Visio. Okay. It doesn't matter. As long as you are comfortable with the tools that you used before, you can keep using. If you are not, then you need to learn something new.

ER Diagram Symbols: A General Explanation

Mohammad explains the basic symbols used in an ER diagram.

- **Rectangle:** Represents an **Entity**. For the first activity, examples include **Students** and **Courses**. Each entity will typically become one database table.
- **Diamond:** Represents a **Relationship**. This shape is used to connect different entities (tables) with each other.
- **Ellipse:** Represents an **Attribute**. These are the properties or columns of a table (e.g., colors in your table).
- **Simple Line:** Used to connect attributes to an entity, or an entity to a relationship symbol.

 **Mohammad:** Let's have a look at these symbols, a general explanation for your diagram. If you have something like a rectangle, it means entity. In the first activity, you have the students, courses. How many courses? Three courses. It means you should have one table to be able to record this information. Otherwise, how we gonna create a table? So, one table for now we can mention is one entity. Thank you. If you want to connect the tables with each other, then you're gonna use the diamond shape. I'm gonna send this screenshot to your teams later on, you're gonna see. Ellipse is an attribute. Attribute, what does it mean? Colors in your table. This simple line if you want to connect the attributes with your entity or connect your entity with the relational symbols.

Mohammad notes that more complex symbols (like double ellipses for weak attributes or double rectangles for weak entities) will be discussed later. For now, the four main symbols are sufficient for the initial design.

 **Mohammad:** We will discuss later on about two ellipses into each other or two rectangles as a weak attribute or weak entity. Just for now, these four on the top are the main things that we need to use for the design of these ER.

He shares a screenshot of these symbols for the students' reference.

 **Mohammad:** I'm gonna put this screenshot, maybe easier for you to read it on your Pings. As in note. Okay, I shared that same with you.

A Simple ER Diagram Example: Customer and Loan

Mohammad presents a simple ER diagram with two entities.

 **Mohammad:** Have a look at this simple ER. How many entities we have? Two. Customer and loan. Okay? We have two tables. The basic things that we need to know, each entity can be developed by one table for now. We have the customer, for example, you want to get a loan or apply for a loan to get a house or whatever. You want to borrow is the relationship between these two entities.

He acknowledges the presence of more complex symbols (like double lines) but advises students not to worry about them for now.

 **Mohammad:** We are using the weak ones... we will discuss later on. We can't ignore these double ones. Just as it, uh, one rectangle for example here, don't think that we can imagine for now is the bit in the whole project. Then we can, I'll recognize which one is weak, which one not. Thank you.

 **Mohammad:** Okay, can you see through that s...

2. 中文全文翻译

定义实体与ER图

Mohammad 继续讲课，回顾了实体-关系（ER）图的核心概念。

 **Mohammad:** 什么是实体？我们在谈论ER。实体。实体有什么说法吗？活动。我现在先把它放在括号里。活动。你能看到你们小组的第一个活动吗？好的，你们的图，它指的是实体关系图。不用“类”这个词，我可以让它稍微.....我们开始吧。

Mohammad 提出了一种教学方法：先展示一个ER图示例，进行讲解，然后让学生自己设计。

 **Mohammad:** 我将向你们展示一个ER图，然后我会解释并讲解这个例子。你们会自动理解它的含义。然后你们就可以为第一个活动进行设计了。

学生关于工具的问题

 **Speaker 3:** 我们可以用什么工具来画图？

👤 **Mohammad**: 我会分享的。网上有一些免费的在线平台。你们可以在上面设计ER图。任何一个都可以。或者如果你的系统里有Visio，你也可以用Visio。好的，这没关系。只要你用着之前用过的工具觉得顺手，就可以继续用。如果你不顺手，那你就需要学点新东西了。

ER图符号：通用解释

Mohammad 解释了ER图中使用的基本符号。

- **矩形**：代表一个**实体**。对于第一个活动，例子包括 **学生** 和 **课程**。每个实体通常会成为数据库中的一个表。
- **菱形**：代表一个**关系**。这个形状用于连接不同的实体（表）。
- **椭圆**：代表一个**属性**。这些是表的属性或列（例如，表中的颜色）。
- **简单线条**：用于将属性连接到实体，或将实体连接到关系符号。

👤 **Mohammad**: 让我们看看这些符号，这是对你们图的通用解释。如果你有一个像矩形的东西，它代表实体。在第一个活动中，你们有学生、课程。有多少门课程？三门课程。这意味着你应该有一个表来记录这些信息。否则我们怎么创建表呢？所以，现在我们可以说一个表就是一个实体。谢谢。如果你想把这些表相互连接起来，那么你就要用到菱形。稍后我会把这个截图发到你们的Teams群里，你们会看到的。椭圆是属性。属性是什么意思？就是表中的列。这条简单的线，如果你想将属性与实体连接起来，或者将你的实体与关系符号连接起来。

Mohammad 指出，更复杂的符号（如表示弱属性的双椭圆或表示弱实体的双矩形）将在后面讨论。目前，这四个主要符号对于初始设计来说已经足够了。

👤 **Mohammad**: 我们稍后会讨论两个椭圆套在一起，或者两个矩形作为弱属性或弱实体的情况。就目前而言，顶部的这四个是我们设计这些ER图需要用到的主要东西。

他分享了一张这些符号的截图供学生参考。

👤 **Mohammad**: 我会把这个截图发出来，也许你们在Pings上读起来更方便。作为笔记。好的，我已经和你们分享了。

一个简单的ER图示例：客户与贷款

Mohammad 展示了一个包含两个实体的简单ER图。

👤 **Mohammad**: 看看这个简单的ER图。我们有多少个实体？两个。客户和贷款。好吗？我们有两个表。我们需要知道的基本事情是，目前每个实体可以由一个表来开发。我们以

客户为例，比如你想获得贷款或申请贷款来买房或其他什么。你想借款就是这两个实体之间的关系。

他承认存在更复杂的符号（如双线），但建议学生目前不要担心这些。

 **Mohammad**: 我们用的是弱的那种.....我们稍后会讨论。我们不能忽略这些双线。就像它，呃，比如这里的一个矩形，不要认为我们现在可以想象它是整个项目的一部分。然后我们可以，我会识别出哪个是弱的，哪个不是。谢谢。

 **Mohammad**: 好的，你能通过那个s看到.....

Here is the processed content based on the provided subtitle transcript, following all specified requirements.

Professional Software Engineering - Lecture Transcript (Processed)

1. ENTITY RELATIONSHIP TYPES: ONE-TO-ONE, ONE-TO-MANY, MANY-TO-MANY

The lecturer continues the discussion on entity relationships, focusing on cardinality and the number of tables required for different relationship types.

1.1. Defining Relationship Cardinality

- The relationship between a **Customer** and a **Loan** is examined.
- From the **Customer** side: one customer can apply for multiple loans (one-to-many).
- From the **Loan** side: generally, one loan belongs to one customer (one-to-one).
- The lecturer clarifies that this is a general rule, but scenarios like joint accounts can change the cardinality.

 **Mohammad**: It means the type of relationship between the entity. How many loans that you can apply through the bank?  **Student**: Two.  **Mohammad**: One, two. Please come from two. You cannot apply for three loans?  **Speaker 3**: We can apply by people loans.  **Mohammad**: Yeah, as many as you need. I don't want to talk about how many of them will be approved. I'll talk about how many applications for the bank that you can send? As many as you need.

- The lecturer summarizes the cardinality for the Customer-Loan relationship:

- **Customer to Loan:** One-to-Many (one customer can have multiple loans).
- **Loan to Customer:** One-to-One (one loan belongs to one customer).
- The final relationship is a **One-to-Many** from the Customer's perspective.
- The lecturer acknowledges that the scenario can change this:

 **Mohammad:** Maybe based on the scenario, some of the loans you're going to apply as a joint account. So it depends on how much detail that you need to keep and what is the scenario for that bank.

1.2. Database Table Requirements for Different Relationships

The lecturer explains the number of database tables needed for each relationship type.

- **For One-to-One and One-to-Many relationships:**
 - You need **two tables**: one for the first entity and one for the second entity.
 - This is sufficient to represent the relationship.
- **For Many-to-Many relationships:**
 - You need **three tables**: one for the first entity, one for the second entity, and a third **junction table** (or associative table) to link them.
 - The lecturer asks for an example of a many-to-many relationship.

1.3. Example of a Many-to-Many Relationship

The lecturer uses the example of a **Student** and a **Course** from a previous activity.

- **Student to Course:** One student can take more than one course (One-to-Many).
- **Course to Student:** One course can be taken by more than one student (One-to-Many).
- **Final Relationship:** Because both directions are one-to-many, the overall relationship is **Many-to-Many**.

 **Mohammad:** Okay, let's go back to your activity one is still there. Can take more than one course. How many courses you have? It means one too many. From the other side, from course, one course can be taken by many, more than one. It means one too many from that direction. And then at the end, it means many to many. Okay, for the activity that I share with you for that section, just the relationship between course and a student, so many to many. Because one student can take more than one course, one course can be taken with more than one. For this type of relationship, they call it many to many. Is it clear?

中文全文翻译

专业软件工程 - 讲座文稿（已处理）

1. 实体关系类型：一对一、一对多、多对多

讲师继续讨论实体关系，重点讲解基数以及不同关系类型所需的表数量。

1.1. 定义关系基数

- 考察了**客户**和**贷款**之间的关系。
- 从**客户**方来看：一个客户可以申请多笔贷款（一对多）。
- 从**贷款**方来看：通常，一笔贷款属于一个客户（一对一）。
- 讲师澄清说，这是一般规则，但像联名账户这样的场景可能会改变基数。

 **Mohammad**：它意味着实体之间的关系类型。你可以通过银行申请多少笔贷款？

 **学生**：两笔。  **Mohammad**：一笔，两笔。请从两个开始。你不能申请三笔贷款吗？  **发言人3**：我们可以申请个人贷款。  **Mohammad**：是的，你需要多少都可以。我不想讨论其中有多少会被批准。我要讨论的是你可以向银行发送多少份申请？你需要多少都可以。

- 讲师总结了客户-贷款关系的基数：
 - **客户到贷款**：一对多（一个客户可以有多笔贷款）。
 - **贷款到客户**：一对一（一笔贷款属于一个客户）。
 - 从客户的角度来看，最终关系是**一对多**。
- 讲师承认场景可以改变这一点：

 **Mohammad**：也许基于场景，你申请的某些贷款是联名账户。所以这取决于你需要保留多少细节以及该银行的场景是什么。

1.2. 不同关系的数据库表要求

讲师解释了每种关系类型所需的数据库表数量。

- **对于一对一和一对多关系**：
 - 你需要**两张表**：一张用于第一个实体，一张用于第二个实体。
 - 这足以表示该关系。
- **对于多对多关系**：

- 你需要三张表：一张用于第一个实体，一张用于第二个实体，以及第三张连接表（或关联表）来链接它们。
- 讲师要求举一个多对多关系的例子。

1.3. 多对多关系的例子

讲师使用了之前活动中的学生和课程的例子。

- **学生到课程**：一个学生可以选修多门课程（一对多）。
- **课程到学生**：一门课程可以被多个学生选修（一对多）。
- **最终关系**：因为两个方向都是一对多，所以整体关系是**多对多**。

 **Mohammad**: 好的，让我们回到你的活动一，它还在那里。可以选修多门课程。你有多少门课程？这意味着一个对多个。从另一边，从课程来看，一门课程可以被很多人选修，不止一个。这意味着从那个方向也是一个对多个。然后最终，这意味着多对多。好的，对于我与你分享的那部分活动，就是课程和学生之间的关系，所以是多对多。因为一个学生可以选修多门课程，一门课程可以被多个学生选修。对于这种类型的关系，他们称之为多对多。清楚了吗？

好的，这是根据您的要求，对提供的字幕文本进行处理后的结果。

1. ZERO INFORMATION LOSS & STRUCTURAL REORGANIZATION (ENGLISH ONLY)

Mohammad continues the lecture on database relationships.

He explains that a "many-to-many" relationship exists between courses and students. This is because one student can take multiple courses, and one course can be taken by multiple students.

 **Student**: Is it clear?

Mohammad then moves on to the next step: creating tables. Before creating a table, you must first add attributes to each entity.

- If some attributes are common or similar between two entities, you can place them in the middle, within the relationship table, to avoid redundancy.
- This creates a unique set of attributes for the relationship itself.

When connecting two tables, you use keys. The lecture will cover primary keys, foreign keys, and more later. For now, the focus is on the three main relationship types between entities:

1. **One-to-One**
2. **One-to-Many**
3. **Many-to-Many**

Mohammad asks for an example of a **one-to-one** relationship.

He uses the concept of marriage as an example, but notes that in a real-world database design (e.g., for a city hall), one person might have more than one record. This makes it a **one-to-many** relationship from one side, which violates the condition for a strict **one-to-one**.

 **Student:** One student and one registration number.  **Mohammad:** I'm talking about two entities. You tell me the relationship between two entities. You are talking about the registration. It means we're going to have a registration table and a student information table. You are thinking something like that. One record of registration can be connected with one student because, as your friend mentioned, the student ID is unique. That's why, if you want to design it this way, yes, one record from the registration table will be connected with one student record, and vice versa. One student record will be connected to just one record in the registration table.

He acknowledges that in a real scenario, this might be more complex due to normalization (first, second, etc.), but for now, the goal is to keep it simple to develop the first step. Complexity will be addressed later.

Summary of what was learned:

1. **Model Relationships:** The relationship between entities can be **one-to-one**, **one-to-many**, or **many-to-many**.
2. **Table Development:** After creating an ER (Entity-Relationship) map of the database, the next step is to create the tables.
 - If the relationship between two entities is **one-to-one** or **one-to-many**, you need **two tables** (one for each entity).
 - If the relationship is **many-to-many**, you need **three tables**. The third table is for the relationship itself.

He notes that in the industrial world, this structure is sometimes not strictly followed, and you might not even recognize it; you just have the database.

2. FULL CHINESE TRANSLATION (中文全文翻译)

穆罕默德继续讲解数据库关系。

他解释说，“多对多”关系存在于课程和学生之间。这是因为一个学生可以选修多门课程，而一门课程也可以被多个学生选修。

👤 学生：清楚了吗？

然后，穆罕默德进入下一步：创建表。在创建表之前，必须首先为每个实体添加属性。

- 如果某些属性在两个实体之间是共同的或相似的，可以将它们放在中间的关系表中，以避免冗余。
- 这样就为关系本身创建了一组唯一的属性。

连接两个表时，需要使用键。课程稍后会介绍主键、外键等更多内容。目前，重点是实体之间的三种主要关系类型：

1. 一对一
2. 一对多
3. 多对多

穆罕默德要求举一个一对一关系的例子。

他以婚姻的概念为例，但指出在现实世界的数据库设计中（例如，为市政厅设计），一个人可能有多个记录。这使得它从一侧看是一对多关系，这违反了严格一对一的条件。

👤 学生：一个学生和一个注册号。👤 穆罕默德：我说的是两个实体。你告诉我两个实体之间的关系。你说的是注册。这意味着我们将有一个注册表和一个学生信息表。你是这么想的。一条注册记录可以连接到一个学生，因为正如你同学提到的，学生ID是唯一的。这就是为什么，如果你想这样设计，是的，注册表中的一条记录将连接到一个学生记录，反之亦然。一个学生记录将只连接到注册表中的一条记录。

他承认，在现实场景中，由于规范化（第一范式、第二范式等），这可能会更复杂，但现在的目标是保持简单，以便完成第一步。复杂性将在后面处理。

所学内容总结：

1. 模型关系： 实体之间的关系可以是一对一、一对多或多对多。
2. 表开发： 在创建了数据库的ER（实体-关系）图之后，下一步是创建表。

- 如果两个实体之间的关系是一对一或一对多，则需要**两个表**（每个实体一个）。
- 如果关系是多对多，则需要**三个表**。第三个表用于表示关系本身。

他指出，在工业界，有时并不严格遵守这种结构，甚至可能无法识别出来；你只是拥有这个数据库。

Here is the processed content based on the provided subtitle text, following all specified requirements.

DATABASE DESIGN AND PROFESSIONAL ETHICS

The Core Principle: Table Creation from ER Diagrams

The lecturer continues the discussion on database design, focusing on the practical step of creating tables from an Entity-Relationship (ER) diagram.

- **Determining the Number of Tables:** The number of tables required is directly determined by the relationship between entities.
 - If the relationship between two entities is **one-to-one** or **one-to-many**, you need **two tables** (one for each entity).
 - If the relationship is **many-to-many**, you need **three tables**. The third table is specifically for representing the relationship itself.
- **Real-World Industrial Context:** The lecturer notes that in the industrial world, the initial database design is not always perfect or even recognizable.
 - You might inherit a database where the original design structure is unclear.
 - Your task might be to add new features or modify an existing application on top of this database.
 - A well-designed database makes it easy for a software developer to update.
 - A poorly designed database forces you to change many parts and introduces significant **redundancy** (extra, unnecessary information) into the database.

The Mindset of a Professional Software Developer

The lecturer shifts focus from technical design to the professional attitude required for long-term success.

- **Long-Term Thinking vs. Short-Term Gains:**

- As a software developer, you must care about the quality of your work. Do not adopt the attitude of "I won't be here in six months or a year."
- This kind of short-sighted attitude will quickly become a problem for you.
- This is especially true in **New Zealand**, which is a small community. Your reputation and connections matter.
- **The Importance of References:**
 - It is normal to change companies after a few months or a year, moving from junior to mid-level, or mid-level to senior.
 - However, your career progression depends on the **quality of your role** and the **references** you receive.
 - In New Zealand, genuine references are crucial. Even friends will not give you a good reference if your work is not good.
- **The Two Pillars of a Good Reference: Soft Skills and Technical Skills:**
 - **Soft skills** include your communication, cooperation, support for your team, and more.
 - **Technical skills** are demonstrated by whether you develop projects in a *clean* way or create something full of mess and unclear code.
- **The Trade-off: Quality vs. Time Management:**
 - You must prioritize meeting deadlines, but you must *never* sacrifice the quality of your work.
 - There is a constant **trade-off** between quality and time management for every task.
 - With practice, you will naturally become faster at development.
 - Quality comes from your **documentation**—make it simple and understandable for everyone.
 - A professional software developer is not defined by writing complex code.

 **Lecturer:** "Professional software developer is not a complex coding."

Activity Transition

The lecturer concludes this section by instructing the class to begin working on an activity.

 **Lecturer:** "Now please design the activity one now based on the example that we share. I think we even mentioned what is the relationship between how many entities that you have in your first activity. What is the tools? I think one of your friends asked or your classmate, I'm going to share the..."

数据库设计与职业素养

核心原则：从ER图创建表

讲师继续讨论数据库设计，重点是从实体-关系（ER）图创建表的实际步骤。

- **确定表的数量：** 所需表的数量直接由实体之间的关系决定。
 - 如果两个实体之间的关系是一对一或一对多，则需要**两个表**（每个实体一个）。
 - 如果关系是多对多，则需要**三个表**。第三个表专门用于表示关系本身。
- **现实工业背景：** 讲师指出，在工业界，最初的数据库设计并不总是完美的，甚至可能无法识别。
 - 你可能会接手一个数据库，其原始设计结构不清晰。
 - 你的任务可能是在这个数据库之上添加新功能或修改现有应用程序。
 - 设计良好的数据库使软件开发者易于更新。
 - 设计不佳的数据库会迫使你修改许多部分，并在数据库中引入大量的**冗余**（额外的、不必要的信息）。

专业软件开发者的心态

讲师将重点从技术设计转向长期成功所需的专业态度。

- **长期思维 vs. 短期利益：**
 - 作为一名软件开发者，你必须关心你的工作质量。不要抱有“我六个月或一年后就不在这里了”的态度。
 - 这种短视的态度很快就会给你带来麻烦。
 - 这在**新西兰**尤其如此，因为这是一个小社区。你的声誉和人脉非常重要。
- **推荐信的重要性：**
 - 几个月或一年后换公司是很正常的，从初级升到中级，或从中级升到高级。
 - 然而，你的职业发展取决于你**角色的质量**以及你获得的**推荐信**。
 - 在新西兰，真实的推荐信至关重要。如果你的工作做得不好，即使是朋友也不会给你好的推荐。
- **优秀推荐信的两大支柱：软技能和硬技能：**
 - **软技能**包括你的沟通能力、合作能力、对团队的支持等等。
 - **硬技能**则体现在你是以**干净的方式**开发项目，还是制造出一堆混乱、不清晰的代码。
- **权衡：质量与时间管理：**
 - 你必须优先考虑按时完成任务，但**绝不能**牺牲工作质量。
 - 对于每项任务，在**质量**和**时间管理**之间都存在持续的**权衡**。
 - 通过练习，你自然会变得开发速度更快。

- 质量来自于你的**文档**——让它简单易懂，让每个人都能理解。
- 一个专业的软件开发者不是以编写复杂代码来定义的。

 讲师：“专业的软件开发者不是编写复杂的代码。”

活动过渡

讲师在结束本节内容时，指示全班开始进行一项活动。

 讲师：“现在，请根据我们分享的例子来设计第一个活动。我想我们甚至提到了你的第一个活动中有多少个实体以及它们之间的关系。用什么工具？我想你的一个朋友或同学问过，我将要分享...”

Professional Software Engineering - Lecture Transcript (Processed)

Activity 1: Database Design Task

TASK INSTRUCTIONS

The lecturer instructs the class to begin designing their first database activity.

 **Lecturer:** "Now please design activity one now based on the example that we share. I think we even mentioned what is the relationship between how many entities that you have in your first activity."

RECOMMENDED TOOLS

The lecturer provides tool recommendations for designing diagrams:

- **Online platform:** A link will be shared for free online resources
- **Desktop software:** Microsoft Visio can be installed on your system
- **Multi-purpose tool:** The recommended tool supports not only database design but also:
 - UML diagrams
 - Use case diagrams
 - And many other diagram types

 **Lecturer:** "I would prefer to share with you more online resources, then you can use it freely to design. It's not only for designing the database. Later on, you want to design the UML diagram, use cases are many more, then you can use it still those tools."

TEMPLATE USAGE

 **Lecturer:** "The link that I shared, there are lots of templates. It's already the template can help you to save the time. You can just modify it using the for example visual right there."

ACTIVITY REQUIREMENTS

The lecturer specifies the minimum requirements for the first activity:

- **Entities:** Student and Course
- **Relationship:** Many-to-Many (M to N)
- **Attributes:**
 - For Courses: minimum 2-3 attributes
 - For Enrollment: some attributes

 **Lecturer:** "For now, I think you have a student, you have course, and you're gonna possibly have relationship like M to N, many to many, and you're gonna add the attributes, couple of attributes for the courses, minimum two or three, and then for courses and some for maybe enrollment."

DESIGN VS. DEVELOPMENT CLARIFICATION

 **Lecturer:** "We don't want to develop the database for the first activity, just we want to design."

TIME ALLOCATION

 **Lecturer:** "From now, 10 minutes."

DOCUMENTATION REQUIREMENTS

The lecturer emphasizes the importance of explaining the design rationale:

 **Lecturer:** "When you are designing, possibly is the good thing to explain in the README file. Because I haven't discussed about the keys, we will talk soon. What are the different type of keys? But for the ER diagram, please explain what you are thinking. It means you're gonna write the scenario behind your ER. I want to understand what you are thinking that you are designing."

DESIGN FLEXIBILITY

 **Lecturer:** "Because later on, for one project, for one simple project, you can design a different, a little bit different ER. Based on the scenario for the project. That's why I'm asking you to explain what you are thinking, and then the ER should be matched or aligned with your scenario and ER should be aligned with each other. Just ER is okay for the first step but please add your explanation."

STUDENT TECHNICAL ISSUE

 **Student:** "Hi Doctor, I think I got a network problem, maybe I'm trying to push my picture to the GitHub failed and it will be that network problem timeout."

 **Lecturer:** "Education Wi-Fi? Do you have access to the education Wi-Fi?"

 **Student:** "Yeah."

 **Lecturer:** "Do you think it doesn't work? SSH failed too? GitHub. Can you push?"

 **Lecturer:** "Maybe anyone else? You can do it later on. Just you can put a screenshot of your outcome."

中文全文翻译

活动一：数据库设计任务

任务说明

讲师指示全班开始设计他们的第一个数据库活动。

 **讲师：** "现在请根据我们分享的例子来设计第一个活动。我想我们甚至提到了你的第一个活动中有多少个实体以及它们之间的关系。"

推荐工具

讲师提供了设计图表时的工具推荐：

- **在线平台：**将分享一个链接，提供免费在线资源
- **桌面软件：**可以在系统上安装微软 Visio
- **多功能工具：**推荐的工具不仅支持数据库设计，还支持：
 - UML 图
 - 用例图
 - 以及许多其他类型的图表

 **讲师：** "我更倾向于与你们分享更多的在线资源，然后你们可以免费使用它们进行设计。它不仅仅用于设计数据库。以后，当你们想要设计 UML 图、用例图等更多内容时，你们仍然可以使用这些工具。"

模板使用

 **讲师：** "我分享的链接里有很多模板。这些模板可以帮助你们节省时间。你们可以直接修改它，比如使用那里的可视化工具。"

活动要求

讲师明确了第一个活动的最低要求：

- **实体：**学生和课程
- **关系：**多对多（M 对 N）
- **属性：**
 - 课程：至少 2-3 个属性
 - 选课关系：一些属性

 **讲师：** "目前，我认为你们有学生实体和课程实体，你们可能会建立像 M 对 N 这样的多对多关系，然后你们需要添加属性，课程至少两到三个属性，选课关系也需要一些属性。"

设计与开发的区别说明

 **讲师：** "第一个活动我们不需要开发数据库，只需要进行设计。"

时间分配

 讲师: "从现在开始, 10分钟。"

文档要求

讲师强调了解释设计思路的重要性:

 讲师: "当你们进行设计时, 最好在 README 文件中进行解释。因为我还没有讨论过键的问题, 我们很快就会讲到。不同类型的键有哪些? 但对于 ER 图, 请解释你们的想法。这意味着你们需要写出 ER 图背后的场景。我想理解你们在设计时的思考过程。"

设计的灵活性

 讲师: "因为以后, 对于一个项目, 一个简单的项目, 你们可以根据项目的场景设计出略有不同的 ER 图。这就是为什么我要求你们解释你们的想法, 然后 ER 图应该与你们的场景相匹配或对齐, ER 图之间也应该相互对齐。第一步只画 ER 图是可以的, 但请加上你们的解释。"

学生技术问题

 学生: "博士您好, 我遇到了网络问题, 我尝试将图片推送到 GitHub 失败了, 可能是网络超时的问题。"

 讲师: "教育 Wi-Fi? 你能连上教育 Wi-Fi 吗?"

 学生: "能连上。"

 讲师: "你觉得它不管用吗? SSH 也失败了吗? GitHub。你能推送吗?"

 讲师: "其他人也有这个问题吗? 你可以稍后再做。只需提交你的结果截图即可。"

Professional Software Engineering - Session Transcript (Continued)

Course Administration & Technical Support

NETWORK AND ACCESS ISSUES RESOLUTION

The lecturer addressed a student's network connectivity problem:

 Student: "It will be that network problem timeout."

 **Lecturer:** "Education Wi-Fi? Do you have access to the education Wi-Fi?"

 **Student:** "Yeah."

 **Lecturer:** "Do you think it doesn't work? SSH failed too? GitHub. Can you push?"

 **Lecturer:** "Maybe anyone else? You can do it later on. Just you can put a screenshot of your outcome with a description directly to your teams."

TEAMS PLATFORM ACCESS

The lecturer provided instructions for submitting work through Microsoft Teams:

- **Direct submission:** Students can send their work directly to Teams with a screenshot and description.
- **Element invitation:** The lecturer will add students through the Elements platform.
- **Platform check:** Students are asked to check their Elements account for access.

 **Lecturer:** "Do you have access to the teams?"

 **Student:** "Yeah."

 **Lecturer:** "Okay. For now, just you can send it directly. I'll add you through the elements. Please check your elements as well. Okay. Hopefully you're going to have access."

MULTILINGUAL INTERACTION

The session included brief exchanges in multiple languages:

- **Chinese (Mandarin):** "网站" (website), "我去看" (I'll go check)
- **Japanese:** "おやすみなさい" (good night), "ありがとうございました" (thank you very much)
- **Chinese (Mandarin):** "谢谢大家" (thank you everyone)
- **English:** "Thank you."

ASSIGNMENT SUBMISSION STATUS

 **Lecturer:** "Okay, do you need more time? What's going on? You haven't done, you haven't finished?"

 **Student:** "Yeah, please."

 **Lecturer:** "Please share on the teams to reply. So far just two. The last one is Eric. Just a second. I checked. Eric, where are you?"

FEEDBACK ON STUDENT WORK (ER DIAGRAM DESIGN)

The lecturer provided specific feedback on a student's database design submission:

 **Lecturer:** "Can I show, there is a little bit of modification that you can add. It's quite good, just this arrow should be as a simple line, instead of the arrow. And course ID possibly are talking about the primary care foreign keys. Okay. Here we have the student name. The rest is fine for now. Just leave these things as a simple line to connect it."

DATABASE DESIGN RESOURCES

The lecturer announced the sharing of learning materials:

 **Lecturer:** "I'm gonna share an ebook, then you can have access to be able to learn more about PR designing. It means to design your database and optimization your database."

ORACLE PLATFORM ACCESS

 **Lecturer:** "Do you have access to Oralee or not yet? I cannot show it to you. Maybe it's easier not to get lost."

 **Student:** "I'll see you next time."

 **Lecturer:** "Okay."

ORACLE LEARNING PLATFORM DETAILS

The lecturer explained the Oracle educational platform:

- **Platform nature:** A comprehensive learning resource that all Master's students should eventually access.
- **Cost:** The platform is not free, but students do not need to pay personally.
- **Access method:** Students can log in using their student email address.
- **Available resources:** Full range of materials including e-books, short video tutorials, and more.
- **Scope:** Resources are available not just for this course but for other courses as well.

- **First-time login:** Students may need to change their password upon initial login.
- **Troubleshooting:** If login fails, students should contact the appropriate administrator.

 **Lecturer:** "It's a platform that all of you should have access sooner or later. Or I'll leave. Of course, it's not free. This platform, but you can learn in using your student email address. And I will share a couple of e-books. There are full of resources, including the e-books, short video, tutorial, and many more. As long as you are the master student then you should have access. That means we want to have access to this platform with your student email address. Okay, does it make sense? Okay, orally then you can download you can read the different resources for the not just this course for other courses. If it's the first time that you try to log in, maybe you need to log in and change your password. If you cannot log in then you need to ask your possibility. I'm gonna give you access because then you should have the admin access. I don't have that many like this, they will invite you for that. It's not free, that's why you don't need to pay. College a..."

中文全文翻译

专业软件工程 - 课程记录（续）

课程管理与技术支持

网络与访问问题解决

讲师回应了学生遇到的网络连接问题：

 **学生：** "是那个网络超时的问题。"

 **讲师：** "教育 Wi-Fi? 你能连上教育 Wi-Fi 吗? "

 **学生：** "能连上。"

 **讲师：** "你觉得它不管用吗? SSH 也失败了吗? GitHub。你能推送吗? "

 **讲师：** "其他人也有这个问题吗? 你可以稍后再做。只需将你的结果截图连同描述直接提交到 Teams 上。"

TEAMS 平台访问

讲师提供了通过 Microsoft Teams 提交作业的说明：

- **直接提交：** 学生可以直接将作品（含截图和描述）发送到 Teams。
- **Element 邀请：** 讲师将通过 Elements 平台添加学生。
- **平台检查：** 要求学生检查自己的 Elements 账户以获取访问权限。

 讲师："你能访问 Teams 吗？"

 学生："能。"

 讲师："好的。现在，你可以直接发送。我会通过 Elements 添加你。也请检查你的 Elements。好的。希望你能获得访问权限。"

多语言交流

课程中出现了简短的多语言交流：

- **中文（普通话）：** "网站"，"我去看"
- **日语：** "おやすみなさい"（晚安），"ありがとうございました"（非常感谢）
- **中文（普通话）：** "谢谢大家"
- **英语：** "Thank you."

作业提交状态

 讲师："好的，你们还需要更多时间吗？怎么了？你还没做完吗？"

 学生："是的，请再给点时间。"

 讲师："请在 Teams 上分享回复。目前只有两份。最后一个是 Eric。稍等一下。我查一下。Eric，你在哪里？"

学生作业反馈（ER 图设计）

讲师对学生的数据库设计提交给出了具体反馈：

 讲师："我可以展示一下，这里有一点可以修改的地方。整体很不错，只是这个箭头应该改成简单的线条，而不是箭头。还有课程 ID 可能指的是主键外键的问题。好的。这里我们有学生姓名。其余部分目前没问题。只需把这些连接线改成简单的线条即可。"

数据库设计资源

讲师宣布将分享学习资料：

👤 讲师: "我将分享一本电子书, 这样你们就可以学习更多关于 PR 设计的内容。也就是数据库设计和数据库优化。"

ORACLE 平台访问

👤 讲师: "你们能访问 Oracle 了吗? 还不能? 我没办法展示给你们看。也许这样不容易搞混。"

👤 学生: "下次见。"

👤 讲师: "好的。"

ORACLE 学习平台详情

讲师解释了 Oracle 教育平台:

- **平台性质:** 一个全面的学习资源平台, 所有硕士生最终都应能访问。
- **费用:** 该平台并非免费, 但学生无需个人付费。
- **访问方式:** 学生可以使用自己的学生邮箱登录。
- **可用资源:** 包含电子书、短视频教程等丰富的学习材料。
- **适用范围:** 资源不仅适用于本课程, 也适用于其他课程。
- **首次登录:** 学生首次登录时可能需要修改密码。
- **故障排除:** 如果登录失败, 学生应联系相应的管理员。

👤 讲师: "这是一个你们所有人都迟早应该能访问的平台。当然, 这个平台不是免费的, 但你们可以使用学生邮箱学习。我会分享几本电子书。这里有丰富的资源, 包括电子书、短视频、教程等等。只要你们是硕士生, 就应该能访问。也就是说, 我们希望你们能用学生邮箱访问这个平台。好的, 明白了吗? 好的, 那么你们就可以下载和阅读不同的资源, 不仅限于本课程, 也包括其他课程。如果是你们第一次尝试登录, 可能需要登录并修改密码。如果无法登录, 你们需要询问可能性。我会给你们访问权限, 因为你们应该有管理员权限。我没有那么多权限, 他们会邀请你们的。它不是免费的, 所以你们不需要付费。学院..."

好的, 作为严谨的学术助理, 我将严格按照您的要求, 对提供的字幕文件进行处理。

1. ZERO INFORMATION LOSS (ENGLISH ONLY)

The following text is a restructured and cleaned version of the provided transcript. All substantive information regarding course logistics, technical instructions, and academic

resources has been preserved. Only filler words (um, ah), stutters, and non-substantive repetitions have been removed for clarity.

2. STRUCTURAL REORGANIZATION & "BREATHING" LAYOUT (ENGLISH ONLY)

Access to the "Oral Hub" Platform

The lecturer, Mohammad, is explaining how students can gain access to the "Oral Hub" platform, which is a resource for the course.

- The platform is not free, but students do not need to pay for it.
- The college already has a subscription.
- They will add your student ID and email address to grant you access.
- The URL for the platform is: `oral.hub`.

When you log in, you should be able to see your profile and your personal resources. If you don't have access yet, don't worry; you will get it soon.

 **Student:** (Unidentified student responds "Okay.")

 **Mohammad:** Maybe you can help to remind Arun, for example, next week to give access if you don't have it. If one of you has access, most likely the rest should have access. It doesn't mean that if I have access, my classmate next to me shouldn't have it. It's not urgent for now, but thanks to your class representative, we're going to follow up.

- **Current Status Check:** Mohammad asks the class if they currently have access. The response is "No. No. No. Okay."

An Open-Source Ebook for Database Design

While waiting for access to the Oral Hub, Mohammad shares an alternative resource.

- He will show an ebook that is open source and open access.
- This ebook can be used freely for designing ER diagrams and databases.
- The ebook covers topics from the beginning:
 - Database designing
 - Entity-Relationship (ER) diagrams
 - How to map the ER with databases like **SQLite3** or **MySQL** for development.

- The table of contents shows it progresses from simple queries (initial concepts) to advanced ones.
- The last chapter covers the advanced concept of **normalization**.

 **Mohammad:** I'm just going to share this ebook for your self-child learning. Till you're going to have access to the Oral Hub, you can read it.

Interaction on Sharing Designs

A student attempts to share a link to their design, but it is not accessible.

 **Student:** (The link that you share is not accessible through your GitHub.)

 **Mohammad:** Is it your GitHub? Maybe because you already logged in, this link is not accessible for me. Because you are using different tools. Maybe put a screenshot of your design.

Closing Remarks

- **Mohammad:** Any questions for the basic designing the ER?
 - **Mohammad:** There is an open access ebook that I'm going to share with you, the PDF one.
 - **Mohammad:** Okay, do you have access to this ebook? Can you download it?
-

3. LOGICAL HIGHLIGHTING (ENGLISH ONLY)

- **Core Concepts & Tools:** Oral Hub, SQLite3, MySQL, normalization, Entity-Relationship (ER) diagrams.
 - **Secondary Emphasis:** *self-child learning, open access, open source.*
-

4. INTERACTION EXTRACTION (ENGLISH ONLY)

 **Student:** Okay.

 **Mohammad:** Maybe you can help to remind Arun, for example, next week to give access if you don't have it. If one of you has access, most likely the rest should have

access. It doesn't mean that if I have access, my classmate next to me shouldn't have it. It's not urgent for now, but thanks to your class representative, we're going to follow up.

 **Mohammad:** For now, do you have access or not?

 **Students:** No. No. No.

 **Mohammad:** Okay.

 **Student:** (The link that you share is not accessible through your GitHub.)

 **Mohammad:** Is it your GitHub? Maybe because you already logged in, this link is not accessible for me. Because you are using different tools. Maybe put a screenshot of your design.

5. 中文全文翻译 (FULL CHINESE TRANSLATION)

访问“Oral Hub”平台

讲师 Mohammad 正在解释学生如何获得对“Oral Hub”平台的访问权限，该平台是课程的一个资源。

- 该平台并非免费，但学生无需付费。
- 学院已经购买了订阅。
- 他们会添加你的学生证号和电子邮件地址来授予你访问权限。
- 该平台的网址是： `oral.hub` 。

当你登录后，你应该能看到你的个人资料和你的个人资源。如果你还没有访问权限，别担心，你很快就会有的。

 **学生：** （一名未确认的学生回答“好的”。）

 **Mohammad:** 也许你可以帮忙提醒一下 Arun，比如下周，如果你还没有权限的话，让他给你开通。如果你们中有人有权限了，很可能其他人也都有了。这并不意味着如果我有权限，我旁边的同学就没有。这件事目前不紧急，但感谢你们的班长，我们会跟进这件事。

- **当前状态检查：** Mohammad 询问全班同学目前是否有访问权限。回答是“没有。没有。没有。好的。”

一本关于数据库设计的开源电子书

在等待 Oral Hub 访问权限的同时，Mohammad 分享了一个替代资源。

- 他将展示一本开源且开放获取的电子书。
- 这本电子书可以免费用于设计 ER 图和数据库。
- 这本电子书涵盖了从基础开始的主题：
 - 数据库设计
 - 实体-关系 (ER) 图
 - 如何将 ER 图映射到像 **SQLite3** 或 **MySQL** 这样的数据库进行开发。
- 目录显示它从简单查询（初始概念）逐步深入到高级查询。
- 最后一章涵盖了 **规范化** 的高级概念。

 **Mohammad:** 我将分享这本电子书供你们自学。在你们获得 Oral Hub 的访问权限之前，可以先阅读它。

关于分享设计的互动

一名学生试图分享一个他们设计的链接，但该链接无法访问。

 **学生:** （你分享的那个链接，通过你的 GitHub 无法访问。）

 **Mohammad:** 这是你的 GitHub 吗？可能是因为你已经登录了，这个链接对我来说无法访问。因为你使用了不同的工具。也许可以放一张你设计的截图。

结束语

- **Mohammad:** 关于基本的 ER 设计有什么问题吗？
- **Mohammad:** 我有一本开放获取的电子书要分享给你们，是 PDF 格式的。
- **Mohammad:** 好的，你们能访问这本电子书吗？能下载吗？

好的，作为您的严谨学术助理，我将严格按照您的要求，对这份字幕文件进行处理。

1. STRUCTURED SUMMARY (ENGLISH)

Introduction to Keys and the E-Book

- **Mohammad** confirms that students can access and download the shared e-book.
- He directs students to **Chapter 4** of the e-book, which contains a subsection discussing different types of keys, specifically the **primary key**.
- The goal is to understand the definition of different key types within a diagram.
- **Mohammad** emphasizes the importance of getting used to reading different resources.

Defining the Primary Key

- **Mohammad** asks students to open the e-book to **page 53** to read a short paragraph defining primary keys.
- He also directs them to **page 15** for another short description.
- He suggests searching for “primary keys” to jump to the relevant section.
- **Mohammad** points out a table in the e-book, noting that at the bottom, a **primary key** is written, but it is composed of **two attributes**.
- He clarifies a common misconception: a primary key is not always a single unique attribute; it can be a composite key.

Mohammad: “I wanted to share with you always the primary key is not just one unique attribute.”

The Learning Process: Asking “Why?”

- **Mohammad** explains that learning needs come from asking “WH questions” (Why, Where, etc.).
- He encourages a curious mindset: “If your brain working something like this, always you are curious to search and learn more.”

Selecting the Best Primary Key

- **Mohammad** provides the basic definition: a primary key attribute must be **unique**.
- He poses a problem: what if an entity has more than one attribute with unique values?
- He uses the example of a student entity, where **Student ID** is a unique attribute.
- He asks which attribute is the best choice for a primary key when multiple unique attributes exist.

 **Student (Speaker 4):** “the shorter one may be?”

 **Mohammad:** “Yes, right. Well done. It means if you have two attributes in one table, both of them, the value on that column is unique for each record, you're gonna choose which one is the shorter value.”

- **Mohammad** provides a concrete example: in a student information table, you might have both a **passport number** and a **student number**. Both are unique, but the shorter one (student number) is typically the better choice for a primary key.
-

2. 中文全文翻译 (FULL CHINESE TRANSLATION)

关于键和电子书的介绍

- **Mohammad** 确认学生可以访问并下载共享的电子书。
- 他引导学生查看电子书的**第4章**，其中有一个小节讨论了不同类型的键，特别是**主键**。
- 目标是理解图表中不同键类型的定义。
- **Mohammad** 强调了习惯阅读不同资源的重要性。

定义主键

- **Mohammad** 要求学生打开电子书到**第53页**，阅读一段关于主键定义的简短段落。
- 他还引导他们查看**第15页**的另一段简短描述。
- 他建议搜索“primary keys”以跳转到相关章节。
- **Mohammad** 指出了电子书中的一个表格，并说明在表格底部写着**主键**，但它是由**两个属性**组成的。
- 他澄清了一个常见的误解：主键并不总是一个唯一的属性，它可以是复合键。

Mohammad: “我一直想和大家分享，主键不仅仅是一个唯一的属性。”

学习过程：问“为什么？”

- **Mohammad** 解释说，学习需求来自于问“WH问题”（为什么、在哪里等）。
- 他鼓励一种好奇的心态：“如果你的大脑像这样运作，你总是会好奇地去搜索和学习更多。”

选择最佳主键

- **Mohammad** 给出了基本定义：主键属性必须是**唯一的**。
- 他提出了一个问题：如果一个实体有多个具有唯一值的属性怎么办？
- 他以学生实体为例，其中**学号**是一个唯一属性。
- 他问，当存在多个唯一属性时，哪个属性是主键的最佳选择。

 **学生（发言人4）**: “可能是较短的那个？”

👤 **Mohammad**: “是的，正确。很好。这意味着如果你在一个表中有两个属性，这两个列的值对于每条记录都是唯一的，你会选择那个值较短的。”

- **Mohammad** 提供了一个具体的例子：在一个学生信息表中，你可能同时有**护照号码**和**学号**。两者都是唯一的，但较短的（学号）通常是主键的更好选择。

好的，这是根据您的要求对提供的字幕文本进行处理后的结果。

1. ZERO INFORMATION LOSS & STRUCTURAL REORGANIZATION (ENGLISH ONLY)

Mohammad continues the discussion on database keys, focusing on the practical selection of a primary key and the introduction of foreign keys.

- **Choosing a Primary Key: The Shorter Value**
 - If a table has two attributes, and both columns contain unique values for every record, you should choose the one with the shorter value.
 - **Example:** In a student information table, you might have both a **passport number** and a **student number**.
 - The passport number is likely unique, but it can be 15 characters long, a combination of letters and numbers.
 - The student ID might be 10 characters long, or even 7.
 - For efficiency, the shorter value (the student ID) is the best option for a primary key.
 - Choosing the longer one is not a problem functionally, but the shorter one is more efficient.
- **Composite Primary Keys**
 - In some tables or entities, you cannot find a single attribute with a unique value for every record.
 - In this case, you need to combine two or more attributes together to create a unique value for each record.
 - This is why, in some tables, you will see a primary key that is composed of more than one column (two or more).
 - **Example:** A product table, which you will read about later, is a common case where you might need a composite key.
- **Recap on Primary Keys**
 - A primary key is a unique attribute for a table.
 - For a **Course** table, the **Course ID** is likely a unique value and a good primary key.

- The `Course Name` is not a good choice for a primary key because it can be repetitive (you can have more than one record with the exact same course name).

- **Introduction to Foreign Keys**

- Each table should have one primary key.
- If you pass this primary key as an attribute into another table, that attribute is called a **foreign key**.
- The purpose of a foreign key is to connect tables with each other.
- **Example:** To create a relationship between a `Student` table and a `Course` table, you would use their primary keys.
- **Abbreviation:** `PK` for Primary Key, `FK` for Foreign Key.

- **Keys in Relationships (Associative Entities)**

- For a relationship (the middle connection table in a many-to-many relationship), the minimum attributes should include:
 - One primary key from the `Student` side (which becomes a foreign key in the relationship).
 - One primary key from the `Course` side (which becomes a foreign key in the relationship).
- **Mohammad** then poses a question to the class: "Do we need to define a primary key for the relation itself?"

2. LOGICAL HIGHLIGHTING (ENGLISH ONLY)

- **Core Concepts:**

- **Primary Key (PK)**
- **Foreign Key (FK)**
- **Composite Primary Key**

- **Key Principles:**

- *Efficiency* is a key factor when choosing between two unique attributes.
- A primary key must be *unique*.
- A foreign key is used to *connect* or *relate* tables.

- **Abbreviations:**

- `PK`
- `FK`

3. INTERACTION EXTRACTION (ENGLISH ONLY)

👤 **Mohammad:** "Each table should have one primary key. Do you agree? Each table should have one primary key. If you pass this primary key as an attribute to another table, that attribute will be a... for that table. It's a simple definition to understand for them. They call it foreign key. What is the reason? Because you want to connect the tables with each other. Am I right? For example, a student with a course. You want to have the relationship between these two tables based on their private. They call it foreign key. As an abbreviation PK, usually you can see is a primary key and FK is foreign key. So minimum attribute for the relation for your ER, it should be one primary key from student side. One primary is foreign key from? Course. For the relation. Do we need to define the primary key for the relation? It means for the middle connection or not? I can share one of the e..."

4. FULL CHINESE TRANSLATION

以下是上述英文内容的中文翻译。

1. 零信息丢失与结构重组

Mohammad 继续讨论数据库键，重点是关于主键的实际选择以及外键的介绍。

- **选择主键：较短的值**
 - 如果一个表中有两个属性，并且这两列的值对于每条记录都是唯一的，你应该选择值较短的那个。
 - **例子：** 在一个学生信息表中，你可能同时有**护照号码**和**学号**。
 - 护照号码很可能是唯一的，但它可能有15个字符长，是字母和数字的组合。
 - 学生ID可能是10个字符长，甚至只有7个。
 - 为了提高效率，较短的值（学号）是作为主键的最佳选择。
 - 从功能上讲，选择较长的那个不是问题，但较短的那个效率更高。
- **复合主键**
 - 在某些表或实体中，你无法找到一个属性对于每条记录都有唯一的值。
 - 在这种情况下，你需要将两个或多个属性组合在一起，以便为每条记录创建一个唯一的值。
 - 这就是为什么在某些表中，你会看到一个主键由多个列（两个或更多）组成。
 - **例子：** 一个产品表，你稍后会读到，是一个常见的可能需要复合键的例子。
- **主键回顾**
 - 主键是表的一个唯一属性。
 - 对于一个**课程表**，**课程ID** 很可能是一个唯一的值，是一个很好的主键。

- **课程名称** 不是主键的好选择，因为它可能是重复的（你可以有多个记录具有完全相同的课程名称）。

- **外键介绍**

- 每个表应该有一个主键。
- 如果你将这个主键作为属性传递到另一个表中，那么这个属性被称为**外键**。
- 外键的目的是将表相互连接起来。
- **例子：**要创建一个 **学生** 表和 **课程** 表之间的关系，你会使用它们的主键。
- **缩写：** **PK** 代表主键， **FK** 代表外键。

- **关系中的键（关联实体）**

- 对于一个关系（多对多关系中的中间连接表），最小的属性应该包括：
 - 来自 **学生** 端的一个主键（在关系中成为外键）。
 - 来自 **课程** 端的一个主键（在关系中成为外键）。
- **Mohammad** 然后向班级提出了一个问题：“我们是否需要为这个关系本身定义一个主键？”

2. 逻辑标注

- **核心概念：**

- **主键 (PK)**
- **外键 (FK)**
- **复合主键**

- **关键原则：**

- **效率** 是在两个唯一属性之间进行选择时的关键因素。
- 主键必须是**唯一**的。
- 外键用于**连接或关联表**。

- **缩写：**

- **PK**
- **FK**

3. 师生互动提取

 **Mohammad:** “每个表应该有一个主键。你们同意吗？每个表应该有一个主键。如果你把这个主键作为属性传递到另一个表中，那个属性将成为该表的.....一个。这是一个简单的定义，方便他们理解。他们称之为外键。原因是什么？因为你想把表相互连接起来。我说得对吗？例如，学生和课程。你想基于它们的私有键来建立这两个表之间的关系。他们称之为外键。作为缩写，PK，你通常可以看到它代表主键，FK代表外键。所以，对于你的

ER图中的关系，最小的属性应该包括来自学生端的一个主键。一个主键是来自哪里的外键？课程。对于这个关系，我们需要为这个关系本身定义主键吗？我的意思是，对于中间的那个连接表？我可以分享一个.....”

Professional Software Engineering - Lecture Notes (Continued)

3. Database Design: Primary Keys and Foreign Keys (Continued)

3.1 DEFINING KEYS FOR RELATIONSHIPS

Mohammad explains the key attributes for relationships in Entity-Relationship (ER) diagrams:

- **Primary Key (PK):** A unique identifier for each entity
- **Foreign Key (FK):** A primary key from one entity passed into another entity to create relationships

Key Principle: The minimum attributes for a relationship in an ER diagram should include:

- One primary key from the **student** side
- One primary key from the **course** side (acting as a foreign key)

3.2 THE QUESTION OF PRIMARY KEYS FOR JUNCTION TABLES

 **Student:** Do we need to define a primary key for the relationship itself? I mean, for the middle connection table?

 **Mohammad:** Let me share an example from your development work.

3.3 EXAMPLE: STUDENT-COURSE ENROLLMENT

Mohammad presents a practical example:

Student Entity Attributes:

- **Student ID** (Primary Key)

- Full name (first name + last name)
- Email address
- Date of birth
- *Many more attributes possible*

Course Entity Attributes:

- **Course ID** (Primary Key)
- Course title / Course name
- Credits / Hours

The Relationship (Enrollment):

- Student ID → becomes **Foreign Key** in the enrollment table
- Course ID → becomes **Foreign Key** in the enrollment table

3.4 IS A PRIMARY KEY NECESSARY FOR THE ENROLLMENT TABLE?

 **Mohammad:** It depends on the scenario. It is **optional** for now because you are developing a small scope of a large project.

Scenario Analysis:

- **Small scope:** Just student and course → primary key for enrollment is optional
- **Large scope:** Full college database with multiple tables → primary key may become necessary

Real-world example from Master of Software Engineering:

 **Mohammad:** In our program, we don't have this major in other campuses. But imagine a college delivering masters across different campuses:

- Christchurch campus
- Auckland campus
- Online campus
- Other campuses

Extended Scenario:

- Students come from different campuses
- Courses are delivered from different campuses
- Enrollment should track which campus the student is from

Solution:

- Add a **campus name** attribute to the enrollment (junction) table
- Remove campus information from both student and course entities
- The campus attribute belongs to the **relationship**, not to either entity individually

👤 **Mohammad**: Some attributes are repetitive from both entities. For example, campus name. I don't want to know just where students come from; I want to know which campus this course is delivering from. Students can log in online or attend face-to-face in different scenarios.

中文全文翻译

专业软件工程 - 课堂笔记（续）

3. 数据库设计：主键与外键（续）

3.1 为关系定义键

Mohammad 解释了实体关系（ER）图中关系的关键属性：

- **主键（PK）**：每个实体的唯一标识符
- **外键（FK）**：从一个实体传递到另一个实体以创建关系的主键

关键原则：ER图中关系的最小属性应包括：

- 来自学生端的一个主键
- 来自课程端的一个主键（作为外键）

3.2 关于连接表主键的问题

👤 **学生**：我们需要为关系本身定义主键吗？我的意思是，对于中间的连接表？

 **Mohammad**: 让我从你们的开发工作中分享一个例子。

3.3 示例：学生-课程注册

Mohammad 展示了一个实际示例：

学生实体属性：

- 学生ID（主键）
- 全名（名 + 姓）
- 电子邮件地址
- 出生日期
- 可能还有更多属性

课程实体属性：

- 课程ID（主键）
- 课程标题 / 课程名称
- 学分 / 学时

关系（注册表）：

- 学生ID → 成为注册表中的外键
- 课程ID → 成为注册表中的外键

3.4 注册表是否需要主键？

 **Mohammad**: 这取决于具体场景。目前来说是可选的，因为你正在开发的是一个大型项目的小范围部分。

场景分析：

- 小范围：只有学生和课程 → 注册表的主键是可选的
- 大范围：包含多个表的完整大学数据库 → 主键可能变得必要

来自软件工程硕士的真实案例：

 **Mohammad**: 在我们的项目中，其他校区没有这个专业。但想象一下，一个大学在不同校区开设硕士课程：

- 基督城校区
- 奥克兰校区
- 在线校区
- 其他校区

扩展场景：

- 学生来自不同的校区
- 课程在不同的校区授课
- 注册信息应记录学生来自哪个校区

解决方案：

- 在注册（连接）表中添加**校区名称**属性
- 从学生和课程实体中移除校区信息
- 校区属性属于**关系**，而不属于任何一个单独的实体

 **Mohammad**：有些属性在两个实体中是重复的。例如，校区名称。我不只是想了解学生来自哪里；我想知道这门课程是从哪个校区授课的。学生可以在不同场景下在线登录或面对面参加课程。

好的，作为您的严谨学术助理，我已根据您的要求，对提供的字幕文件进行了处理。

PROFESSIONAL SOFTWARE ENGINEERING - LECTURE TRANSCRIPT (PROCESSED)

1. Course Activity: Entity-Relationship (ER) Diagram Refinement

1.1. Resolving Redundant Attributes

- The lecturer, **Mohammad**, continues the discussion on refining the ER diagram.
- He points out that the **campus name** attribute was previously present in both the **Student** and **Course** entities.
- The solution is to remove this attribute from both entities and add it to the **Enrollment** relationship entity.
- This is because the campus is a property of the enrollment instance (a specific student taking a specific course), not a property of the student or the course alone.

 **Mohammad:** ...and also we have the campus name here, then you can remove it from both sides and add it to the middle. The campus name is for that course, and I don't want to know just where the students come from; I want to know which campus this course is delivering from. For example, they can log in online or face-to-face in different scenarios. Some attributes are repetitive from both entities, then you can just remove it from the side and add it into the middle relationship.

1.2. Contextual Application of Design Principles

- **Mohammad** clarifies that this is a general principle but must be applied case-by-case.
- The decision to add an attribute to a relationship depends on the specific scenario and requirements.
- For the current activity, if the campus name is not a required attribute, it can be omitted for now.

 **Mohammad:** But for now, if we don't have it, we will not consider it. If we want to expand it, possibly in some scenario, the campus name can be added as one attribute for enrollment, not all the time. Okay, case by case is different.

1.3. Activity 1, Part 2: Adding a New Entity

- **Mohammad** introduces the second part of the current activity.
- The task is to add a new entity, such as `Lecturer` or `Professor` or `Teacher`, to the existing ER diagram.
- The goal is to update the diagram to show the **primary keys** and **foreign keys** for all entities.

 **Mohammad:** If you want to add one more entity like lecturer or professor or whatever, or teacher, for the next activity of course, we want to show the primary key and foreign key, plus add one more entity. Teacher or lecturer, whatever, and update your ER. Okay, do it now as a second part of your activity one. I will not name it as activity two; it's still activity one, part two.

1.4. Clarifying Activity Naming

- A student asks for clarification on the activity naming.
- **Mohammad** confirms that the first part of the activity is now called `Activity 1.1`, and this new part is `Activity 1.2`. They are still part of the same overall activity, just an update.

 **Mohammad:** I already changed the title of the first activity, activity 1.1, and then the next activity should be 1.2. Still, we are in the same activity; just we want to update it.

1.5. Detailed Instructions for Part 2

- **Mohammad** provides more specific instructions for the update.
- He uses a shorthand: **A1** in W3 (the course platform) means **W3 Activity 1**.
- The core task is to update the ER diagram from Part 1 by adding one or more tables for development.
- **Part 1** required three tables: **Student**, **Course**, and **Enrollment** (the relationship).
- **Part 2** requires adding one more entity: **Lecturer**.

 **Mohammad:** From now on, whenever I'm gonna write in W3, for example A1, it means W3 activity 1 is a short form. I just ask: update your ER that is called from part 1 activity 1. Just we want to add one more table or maybe more tables for development. For part one, how many tables that you need? For the development, three. Okay. A student, course, and enrollment. It means the relationship between these two. Please add one more entity as a lecturer, then update your entity.

1.6. Warning: Avoiding Design Traps

- **Mohammad** issues a warning about common design pitfalls.
- The key is to avoid creating **loops** in the ER diagram.
- A loop occurs when relationships between entities create a circular dependency, which is an inefficient and problematic design.

 **Mohammad:** Careful about the traps. That means no loop in your ER. We have different types of traps in your designing or any design.

2. Student Interaction: Foreign Key Clarification

- A student asks a question about the implementation of foreign keys in the new design.

 **Student:** So, there is a requirement that it should have a foreign key. So, if you want to have a foreign key in that table, then you have to create another table.

- **Mohammad** clarifies that the current focus is on the **design** phase (the ER diagram), not the **development** phase (creating actual database tables).
- The question about foreign keys is a development concern.
- For the design phase, the task is simply to add the **Lecturer** entity.

- **Mohammad** hints at the logical connection: the **Lecturer** entity will likely be connected to the **Course** entity, as this is more meaningful and efficient.

👤 **Mohammad**: Talking about the table is the next step for the development. Just for the designing, we are not considering how we gonna have the top view for the development. But to answer your question, here we want just add one more entity as a lecturer. At this lecturer, in the development area, you know when you want to develop any tool, we're gonna have one table for that, and this lecturer can be connected with a student either or course. Mainly with the course, it should be more meaningful, it should be more efficient.

中文全文翻译

专业软件工程 - 讲座文稿（已处理）

1. 课程活动：实体关系（ER）图优化

1.1. 解决冗余属性

- 讲师 **Mohammad** 继续讨论 ER 图的优化。
- 他指出，**校区名称** 属性之前同时存在于 **学生** 和 **课程** 实体中。
- 解决方案是从这两个实体中移除该属性，并将其添加到 **注册** 关系实体中。
- 这是因为校区是注册实例（特定学生选修特定课程）的一个属性，而不是学生或课程本身的属性。

👤 **Mohammad**: ...而且我们这里也有校区名称，那么你可以从两边移除它，并添加到中间。校区名称是针对那门课程的，我不只想知道学生来自哪里；我想知道这门课程是从哪个校区授课的。例如，他们可以在不同的场景下在线登录或面对面参加课程。有些属性在两个实体中是重复的，那么你可以直接把它从一边移除，并添加到中间的关系中。

1.2. 设计原则的上下文应用

- **Mohammad** 澄清说，这是一个通用原则，但必须根据具体情况应用。
- 是否将属性添加到关系中取决于具体的场景和需求。
- 对于当前的活动，如果校区名称不是必需的属性，可以暂时忽略它。

👤 **Mohammad**: 但就目前而言，如果我们没有它，我们就不会考虑它。如果我们想扩展它，可能在某种场景下，校区名称可以作为注册的一个属性添加进来，但并非总是如此。好的，情况各不相同。

1.3. 活动1，第二部分：添加新实体

- **Mohammad** 介绍了当前活动的第二部分。
- 任务是在现有的 ER 图中添加一个新实体，例如 **讲师** 或 **教授** 或 **教师**。
- 目标是更新图表，以显示所有实体的 **主键** 和 **外键**。

 **Mohammad**: 如果你想为课程的下一步活动添加一个实体，比如讲师、教授或其他什么，或者教师，我们想要展示主键和外键，再加上一个实体。教师或讲师，都可以，然后更新你的 ER 图。好的，现在作为你们活动一的第二部分来做。我不会把它命名为活动二；它仍然是活动一，第二部分。

1.4. 澄清活动命名

- 一名学生询问活动命名的澄清。
- **Mohammad** 确认活动的第一部分现在称为 **Activity 1.1**，而这新的一部分是 **Activity 1.2**。它们仍然是同一个整体活动的一部分，只是进行更新。

 **Mohammad**: 我已经更改了第一个活动的标题，活动 1.1，然后下一个活动应该是 1.2。我们仍然在同一个活动中；只是我们想更新它。

1.5. 第二部分的详细说明

- **Mohammad** 提供了更具体的更新说明。
- 他使用了一个简写：在 W3（课程平台）中，**A1** 表示 **W3 活动 1**。
- 核心任务是通过添加一个或多个用于开发的表来更新第一部分的 ER 图。
- **第一部分** 需要三个表：**学生**、**课程** 和 **注册**（关系）。
- **第二部分** 需要添加一个实体：**讲师**。

 **Mohammad**: 从现在开始，每当我在 W3 中写，例如 A1，它意味着 W3 活动 1，这是一个简写形式。我只是要求：更新你们从第一部分活动 1 中得到的 ER 图。我们只是想为开发添加一个或多个表。对于第一部分，你们需要多少个表？对于开发来说，三个。好的。一个学生、一个课程和一个注册。这意味着这两者之间的关系。请再添加一个实体作为讲师，然后更新你的实体。

1.6. 警告：避免设计陷阱

- **Mohammad** 对常见的设计陷阱发出了警告。
- 关键是要避免在 ER 图中创建 **循环**。
- 当实体之间的关系产生循环依赖时，就会发生循环，这是一种低效且有问题的设计。

👤 **Mohammad**: 小心陷阱。这意味着你的 ER 图中不能有循环。在我们的设计或任何设计中都有不同类型的陷阱。

2. 师生互动：外键澄清

- 一名学生询问关于在新设计中实现外键的问题。

👤 **学生**: 所以, 有一个要求是它应该有一个外键。所以, 如果你想在那个表中有一个外键, 那么你必须创建另一个表。

- **Mohammad** 澄清说, 当前的焦点是 **设计** 阶段 (ER 图), 而不是 **开发** 阶段 (创建实际的数据库表)。
- 关于外键的问题是开发层面的关注点。
- 对于设计阶段, 任务只是添加 **讲师** 实体。
- **Mohammad** 暗示了逻辑联系: **讲师** 实体很可能与 **课程** 实体相关联, 因为这更有意义且更高效。

👤 **Mohammad**: 谈论表是开发的下一步。仅仅为了设计, 我们不考虑我们将如何为开发构建顶层视图。但是为了回答你的问题, 这里我们只想添加一个实体作为讲师。对于这个讲师, 在开发领域, 你知道当你想要开发任何工具时, 我们会为它创建一个表, 而这个讲师可以与学生或课程连接。主要与课程连接, 这应该更有意义, 应该更高效。

好的, 作为您的严谨学术助理, 我将严格按照您的要求, 对这份 Level 9 Master of Software Engineering 级别的 Professional Software Engineering 课程视频字幕文件进行处理。

1. STRUCTURAL REORGANIZATION & ZERO INFORMATION LOSS (ENGLISH ONLY)

H2: Entity Relationship Modeling: Course and Lecture

H3: Defining the Relationship

The discussion continues on the Entity-Relationship (ER) diagram. The focus is on adding a **Lecturer** entity and defining its relationship with the **Course** entity.

- The **Lecturer** entity is introduced for the development phase. In a database, this would become a table.

- The **Lecturer** can be connected to a **Student** or a **Course**. Connecting it to **Course** is more meaningful, efficient, and optimized.
- The relationship between **Course** and **Lecturer** needs to be defined as one-to-one, one-to-many, or many-to-many.

 **Mohammad:** ...here we want just add one more entity as a lecture. At this lecture, in the development area you know when you want to develop any tool we're gonna have one table for that and this lecture can be connected with a student either or course mainly with the course it should be more meaningful it should be more efficient and optimized if you're gonna connect it with the course the relationship should be one to one or one to many or many to many.

H3: Determining the Cardinality

The cardinality of the relationship is analyzed.

- One course is delivered by one lecturer.
- One lecturer teaches one course.
- This suggests a many-to-many relationship is needed.

 **Mohammad:** One course is going to be delivered with one lecture from another side. One lecture is going to just teach one course. So it's using many to many. In this case, for the relationship between course and lecture is many to many, that's why you need a gain.

 **Speaker 3:** Considering the course can have multiple lectures, yes, of course there should be a real scenario...

 **Mohammad:** ...yes. Even for your college, yes. I think these are obvious things that one, for example, data analytics, this semester I'm teaching. For the next, you know, for the second, last semester, another lecture, top-down course.

 **Speaker 3:** That's why one course can be delivered on more than one course. And I think even like for individual canister, multiple canister can.

H3: Refining the Entity Attributes

The discussion shifts to the attributes of the **Lecturer** entity.

- The **Lecturer** entity has a relationship with the **Course** name.
- Initial attributes considered: Name, Teacher, and Subject.
- **Subject** is identified as the course name.
- A decision is made to remove **Subject** and **Status**.

- The refined attributes are: Teacher ID, Teacher Name, and Course Name.

 **Speaker 2:** So the lecture only has the relationship with the course name. Yeah. Okay. Five attributes. Name, teacher, and subject. Subject is the course name. I was wondering that too. I think I should remove the subject and the status. Teacher ID, teacher name and course name.

H2: Class Administration and Wrap-up

H3: Attendance and Progress Check

The lecturer checks attendance and asks for student progress.

 **Mohammad:** Meanwhile that you are updating your ER, I cannot check your attendance to save the time. Is that okay? Okay. Thank you. Unbreak call? So, we are... So, is that correct the pronunciation? And here, so it's the same. Do we have? No? I'm checking the attendance, please. Please. I'm turning. Adia? I think I'll get it. Any questions?

 **Speaker 2:** No. I'm done.

 **Mohammad:** No, it means all clear.

 **Speaker 2:** Doing.

 **Mohammad:** Oh, you're doing. Okay. The first activity is done.

 **Speaker 2:** Yes.

 **Mohammad:** Bad bone? Easy? Yeah. Something as a challenge?

 **Speaker 2:** No, no they are all good.

 **Mohammad:** All good. So far so good.

 **Speaker 3:** Yes. Let's see.

 **Mohammad:** Thank you. Bimow. Any question? No.

2. FULL CHINESE TRANSLATION (中文全文翻译)

H2: 实体关系建模：课程与讲师

H3: 定义关系

讨论继续围绕实体关系（ER）图展开。重点是添加一个 **讲师** 实体，并定义其与 **课程** 实体之间的关系。

- 引入 **讲师** 实体是为了开发阶段。在数据库中，这将成为一个表。
- **讲师** 可以连接到 **学生** 或 **课程**。将其连接到 **课程** 更有意义、更高效且更优化。
- **课程** 和 **讲师** 之间的关系需要定义为一对一、一对多或多对多。

 **Mohammad**:这里我们只想添加一个实体作为讲师。对于这个讲师，在开发领域，你知道当你想要开发任何工具时，我们会为它创建一个表，而这个讲师可以与学生或课程连接。主要与课程连接，这应该更有意义，应该更高效、更优化。如果你要将其与课程连接，关系应该是一对一、一对多或多对多。

H3: 确定基数

分析了关系的基数。

- 一门课程由一位讲师授课。
- 一位讲师只教一门课程。
- 这表明需要一个多对多的关系。

 **Mohammad**: 一门课程将由另一位讲师授课。一位讲师只教一门课程。所以它使用的是多对多。在这种情况下，课程和讲师之间的关系是多对多的，这就是为什么你需要一个中间表。

 **Speaker 3**: 考虑到一门课程可以有多个讲师，是的，当然应该有一个真实的场景.....

 **Mohammad**:是的。即使在你的大学里，也是。我认为这些都是显而易见的事情，例如，数据分析，这学期是我在教。下学期，另一个讲师会教这门课。

 **Speaker 3**: 这就是为什么一门课程可以由多个讲师授课。而且我认为即使对于个人容器，多个容器也可以。

H3: 细化实体属性

讨论转向了 **讲师** 实体的属性。

- **讲师** 实体与 **课程** 名称有关系。
- 最初考虑的属性：姓名、教师和科目。
- **科目** 被确定为课程名称。
- 决定移除 **科目** 和 **状态**。
- 精炼后的属性是：教师 ID、教师姓名和课程名称。

 **Speaker 2:** 所以讲师只与课程名称有关系。是的。好的。五个属性。姓名、教师和科目。科目就是课程名称。我也在想这个问题。我认为我应该移除科目和状态。教师 ID、教师姓名和课程名称。

H2: 课堂管理与收尾

H3: 考勤与进度检查

讲师检查考勤并询问学生进度。

 **Mohammad:** 在你们更新 ER 图的同时，为了节省时间，我就不检查考勤了，可以吗？好的。谢谢。通话中断？那么，我们.....那个发音对吗？这里也一样。我们有吗？没有？我正在检查考勤，请。请。我正在点名。Adia？我想我来处理。有什么问题吗？

 **Speaker 2:** 没有。我做完了。

 **Mohammad:** 没有，那就意味着都清楚了。

 **Speaker 2:** 正在做。

 **Mohammad:** 哦，你正在做。好的。第一个活动完成了。

 **Speaker 2:** 是的。

 **Mohammad:** 有困难吗？简单吗？是的。有什么挑战吗？

 **Speaker 2:** 不，不，都很好。

 **Mohammad:** 都很好。到目前为止还不错。

 **Speaker 3:** 是的。让我们看看。

 **Mohammad:** 谢谢。Bimow。有什么问题吗？没有。

Professional Software Engineering - Lecture Interaction & Q&A Session

Course Interaction Log

GENERAL CHECK-IN WITH STUDENTS

 **Mohammad:** Easy?

 **Mohammad:** Yeah.

 **Mohammad:** Something as a challenge?

 **Speaker 2:** No, no they are all good.

 **Mohammad:** All good. So far so good.

 **Speaker 3:** Yes. Let's see.

 **Mohammad:** Thank you. Bimow. Any question?

 **Speaker 3:** No.

 **Mohammad:** Good. Thank you.

ACTIVITY PROGRESS CHECK

 **Mohammad:** Any questions? Any challenge for your activity? Activity one, two, is it easy? You need, is it new concept, the ER for you, or something?

 **Speaker 3:** The, the, the, the, the, the, the, the...

 **Mohammad:** The relation part?

 **Speaker 3:** No, the second activity.

 **Mohammad:** The first one is okay?

 **Speaker 3:** Uh, no.

 **Mohammad:** No fun?

 **Speaker 3:** Yeah.

 **Mohammad:** Any questions?

 **Speaker 3:** Thank you.

INDIVIDUAL STUDENT CHECK-INS

 **Mohammad:** Eric? Pahat? Yes. Any questions? Fun? All good?

 **Mohammad:** Jane Payo? Any questions? Sorry. Sorry, I missed that one. Any questions? Is it a new thing? Is it new for you?

ENCOURAGING STUDENT PARTICIPATION

 **Mohammad:** Okay, I understand. But please ask me, don't feel shy that there are many questions. If you miss it, the first one, never ever you can solve it. You need to of course study during the week, but please ask your question in the class to be able to learn by end of the session and during the week you're gonna study and learn more. Okay?

SPECIFIC ER DIAGRAM QUESTION

 **Mohammad:** For the first activity, for the first ER diagram, is there any question? Why we have two entities for the first one?

 **Speaker 3:** I'm trying to figure out the relationship between this and this and the...

 **Mohammad:** Summer of two classes. You have the research method. It is in the past or this semester? How do you feel about which?

 **Speaker 3:** There are many...

 **Mohammad:** Foreign candidate, many more. Yeah.

 **Speaker 3:** We should input all these information on...

 **Mohammad:** Certain lines. Oh, okay. It's good.

ATTRIBUTES AND PRIMARY KEYS DISCUSSION

 **Mohammad:** Okay. Your friend is asking possibly for each entity we can add many more attributes. Am I right? 10 maybe 20 it depends how much detail that you want to save it in your database but if you can add it two of them or three of them then it should be easy the rest it should be the similar things. If you can find the primary key can we have two primary keys in one table?

 **Mohammad:** No.

 **Mohammad:** Again, can we have two primary keys? For example, we have 20 attributes. Okay, all of the attribute when you are checking maybe five of them are the good candidate. They call it **candidate keys**. But can we have more than one primary key in one...?

中文全文翻译

专业软件工程 - 课堂互动与问答环节

课程互动记录

与学生进行总体情况检查

 **Mohammad:** 简单吗？

 **Mohammad:** 是的。

 **Mohammad:** 有什么挑战吗？

 **Speaker 2:** 不，不，都很好。

 **Mohammad:** 都很好。到目前为止还不错。

 **Speaker 3:** 是的。让我们看看。

 **Mohammad:** 谢谢。Bimow。有问题吗？

 **Speaker 3:** 没有。

 **Mohammad:** 很好。谢谢。

活动进度检查

 **Mohammad:** 有问题吗？活动有什么挑战吗？活动一、活动二，简单吗？你需要，这对你来说是新的概念吗，ER图，还是什么？

 **Speaker 3:** 那个，那个，那个，那个，那个，那个，那个，那个.....

 **Mohammad**: 关系部分?

 **Speaker 3**: 不，是第二个活动。

 **Mohammad**: 第一个没问题吗?

 **Speaker 3**: 呃，不。

 **Mohammad**: 不好玩吗?

 **Speaker 3**: 是的。

 **Mohammad**: 有问题吗?

 **Speaker 3**: 谢谢。

个别学生情况检查

 **Mohammad**: Eric? Pahat? 是的。有问题吗? 好玩吗? 都好吗?

 **Mohammad**: Jane Payo? 有问题吗? 抱歉。抱歉，我漏掉了那个。有问题吗? 这是新东西吗? 这对你来说是新的吗?

鼓励学生参与

 **Mohammad**: 好的，我明白了。但是请问我，不要因为问题多而感到害羞。如果你错过了第一个，你永远也无法解决它。你当然需要在一周内学习，但请在课堂上提问，以便在课程结束时能够学会，然后在一周内你会学习并学到更多。好吗?

关于ER图的具体问题

 **Mohammad**: 对于第一个活动，对于第一个ER图，有问题吗? 为什么第一个图我们有两个实体?

 **Speaker 3**: 我正试图弄清楚这个和这个以及.....之间的关系。

 **Mohammad**: 两个类的总和。你有研究方法。是过去还是这个学期? 你感觉怎么样?

 **Speaker 3**: 有很多.....

 **Mohammad:** 外来的候选者，还有很多。是的。

 **Speaker 3:** 我们应该把这些信息都输入到.....

 **Mohammad:** 某些行上。哦，好的。很好。

关于属性和主键的讨论

 **Mohammad:** 好的。你的朋友可能在问，对于每个实体，我们可以添加更多的属性。我说得对吗？10个或者20个，这取决于你想在数据库中保存多少细节，但如果你能添加两个或三个，那么应该很容易，其余的应该是类似的事情。如果你能找到主键，我们可以在一个表中拥有两个主键吗？

 **Mohammad:** 不。

 **Mohammad:** 再说一遍，我们可以有两个主键吗？例如，我们有20个属性。好的，当你检查所有属性时，也许其中五个是好的候选者。它们被称为**候选键**。但是我们可以有一个表中拥有多个主键吗.....？

好的，作为您的严谨学术助理，我将严格按照您的要求处理这份字幕文件。

Professional Software Engineering - Database Keys & Project Tools

1. The Concept of Primary Keys

1.1 CAN A TABLE HAVE TWO PRIMARY KEYS?

A student asked a fundamental question about database design: can a single table have two primary keys?

 **Student:** If you can find the primary key, can we have two primary keys in one table?

 **Mohammad:** No.

 **Mohammad:** Again, can we have two primary keys? For example, we have 20 attributes. Okay, all of the attributes, when you are checking, maybe five of them are the

good candidates. They call it **candidate keys**. But can we have more than one primary key in one table? Have a look? No. It should be a strong no. One table, one primary key.

1.2 COMPOSITE KEYS VS. MULTIPLE PRIMARY KEYS

Mohammad clarifies the distinction between having multiple primary keys and having a single primary key composed of multiple columns.

 **Mohammad:** Maybe that primary key is the combination of more than one attribute. It means more than one column. For example, two columns. The example that I showed you through the ebooks, that primary key is two columns, am I right? But I see there is one primary key for that table. The scenario behind it: a primary key should be unique. That's why there is one primary key.

1.3 THE PURPOSE OF KEYS: RELATING TABLES

The core reason for having a primary key is to establish relationships between tables.

 **Mohammad:** Because you want to connect it with somewhere else. It means one table to another table. What's the reason that we need the key?

 **Speaker 5:** To relate one table to another table.

 **Mohammad:** Exactly. To relate one table to another table. Because later on you want to join. Some information comes from the first table, some other information comes from the second table, the rest comes from the third table. Then you need to join these, for example, two or three or more tables with each other to be able to... or "collect" is the right word... the information that you want.

1.4 FUTURE APPLICATION: QUERIES

This foundational concept is directly linked to future tasks in the course.

 **Mohammad:** Later on, you're gonna write the different queries to retrieve the information from your database.

1.5 DIFFERENT TYPES OF KEYS

Mohammad addresses the broader question about multiple keys by introducing the concept of different key types, while cautioning against overcomplication at this stage.

 **Mohammad:** That's why the question that came from your friend, it means "can we have more keys?" There are different types of keys. So far, just we will use just **primary key** and **foreign key**. And maybe later on we're going to understand what is the **candidate key**. And there are many more. For the Data Analytics paper, if you want to learn more in detail for the advanced level, then you need to study. If you ask me more, then I will share more. Because I don't want any of you to get confused. You should learn from the first step to make sure your foundation is all good in the same level, and then we can be on top of that.

 **Mohammad:** Okay, did I answer your question, or is there any other question?

2. Project Implementation: Python & SQLite3

2.1 TRANSITIONING TO CODING

The discussion moves from database theory to the practical tools for the course project.

 **Mohammad:** And Python? Oh, Python coding. What was the coding that you used last week? What was the language? Yeah. This week you're gonna learn a new concept and then you're gonna create all of the, develop all of the project using the Python plus other tools like **SQLite3** as a database.

2.2 USING PYTHON TO QUERY THE DATABASE

A student confirms the role of Python in the project.

 **Speaker 3:** We describe information from the database and using the Python.

 **Mohammad:** Yes, we're gonna use Python to be able to write a query and retrieve the data from the table.

3. Class Check-in

Mohammad briefly checks in with the class to ensure everyone is following along.

 **Mohammad:** Let's go with next. Not to Miss Curry. We are here?

 **Student:** Yes, sir.

 **Mohammad:** Where we are? Okay. Any question?

 **Student:** No, sorry. Everything is fine.

 **Mohammad:** All good. Thank you. Mahima?

中文全文翻译

专业软件工程 - 数据库键与项目工具

1. 主键的概念

1.1 一个表可以有两个主键吗？

一位学生提出了一个关于数据库设计的基本问题：一个表能否拥有两个主键？

 **学生：**如果你能找到主键，我们可以在一个表中拥有两个主键吗？

 **Mohammad：**不。

 **Mohammad：**再说一遍，我们可以在一个表中拥有两个主键吗？例如，我们有20个属性。好的，当你检查所有属性时，也许其中五个是好的候选者。它们被称为**候选键**。但是我们可以有一个表中拥有多个主键吗？看看？不。这应该是一个坚决的“不”。一个表，一个主键。

1.2 复合键与多个主键的区别

Mohammad 澄清了拥有多个主键与拥有一个由多个列组成的单一主键之间的区别。

 **Mohammad：**也许那个主键是多个属性的组合。这意味着不止一列。例如，两列。我在电子书中给你们展示的例子中，那个主键就是两列，对吧？但我认为那个表只有一个主键。其背后的场景是：主键必须是唯一的。这就是为什么只有一个主键。

1.3 键的用途：关联表

拥有主键的核心原因是为了在表之间建立关系。

 **Mohammad：**因为你想要把它和别处连接起来。这意味着从一个表到另一个表。我们需要键的原因是什么？

 **发言人5：**为了将一个表与另一个表关联起来。

 **Mohammad:** 完全正确。为了将一个表与另一个表关联起来。因为之后你要进行连接 (JOIN)。一些信息来自第一个表，另一些信息来自第二个表，其余的信息来自第三个表。然后你需要将这些表，例如两个、三个或更多表，相互连接起来，以便能够.....或者“收集”是更准确的词.....你所需的信息。

1.4 未来的应用：查询

这个基础概念直接与课程未来的任务相关联。

 **Mohammad:** 之后，你们将编写不同的查询来从数据库中检索信息。

1.5 不同类型的键

Mohammad 通过介绍不同键类型的概念来回应当关于多个键的更广泛问题，同时告诫现阶段不要过度复杂化。

 **Mohammad:** 这就是为什么你朋友提出的问题，意思是“我们可以有更多的键吗？”有不同类型的键。到目前为止，我们只会用到**主键**和**外键**。也许以后我们会理解什么是**候选键**。还有很多其他的键。对于数据分析课程，如果你想更详细地学习高级内容，那么你需要去学习。如果你问我更多，我会分享更多。因为我不想让你们任何人感到困惑。你们应该从第一步开始学习，确保你们的基础都处于相同的良好水平，然后我们才能在此基础上继续学习。

 **Mohammad:** 好了，我回答你的问题了吗？或者还有其他问题吗？

2. 项目实施：Python 与 SQLite3

2.1 过渡到编码

讨论从数据库理论转向课程项目的实际工具。

 **Mohammad:** 还有 Python？哦，Python 编码。你们上周用的编码是什么？是什么语言？是的。这周你们将学习一个新概念，然后你们将创建所有内容，使用 Python 以及其他工具（如作为数据库的 **SQLite3**）来开发整个项目。

2.2 使用 PYTHON 查询数据库

一位学生确认了 Python 在项目中的作用。

 **发言人3:** 我们使用 Python 来描述来自数据库的信息。

 **Mohammad**: 是的，我们将使用 Python 来编写查询并从表中检索数据。

3. 课堂点名

Mohammad 简短地与班级确认，确保大家都在跟上进度。

 **Mohammad**: 我们继续下一个。不是 Miss Curry。我们在这里吗？

 **学生**: 是的，老师。

 **Mohammad**: 我们在哪里？好的。有问题吗？

 **学生**: 不，抱歉。一切都好。

 **Mohammad**: 很好。谢谢。Mahima？

好的，这是根据您的要求对提供的字幕文本进行处理后的结果。

1. ZERO INFORMATION LOSS & STRUCTURAL REORGANIZATION (ENGLISH)

Mohammad continues taking attendance and checking in with students about their familiarity with database tools.

He asks the class if they are following along.

 **Mohammad**: Let's go with next. Not to Miss Curry. We are here?  **Student**: Yes, sir.

 **Mohammad**: Where we are? Okay. Any question?  **Student**: No, sorry. Everything is fine.  **Mohammad**: All good. Thank you.

He then proceeds to call out individual names.

 **Mohammad**: Mahima?  **Student**: Sir present.  **Mohammad**: All good? 

Student: Yes.  **Mohammad**: And the abu?  **Student**: Yes.  **Mohammad**: Any

question?  **Student**: No.  **Mohammad**: Ming-Doh?  **Student**: Yes. 

Mohammad: How is it going? How is the life? Any questions? Is it a strong no or is it a...

 **Student**: Mingu? No, no, I'm here. Any question? Is it a new concept for you? 

Mohammad: Thank you.

Mohammad continues the roll call.

 **Mohammad:** Next, we are here. Nazmul Shah. Sheikh Abdul Rashid. Nafseh. Neeraj? Neeraj? Neeraj? Any questions? You are familiar with the... Nehima? We are here. Nehima? Any questions?

He then transitions into a discussion about prior knowledge and database tools.

 **Mohammad:** In the undergrad, in the bachelor, you design the database. You learn and all the things. It's the kind of recovery, review all of this. What did you use as a tool in your undergrad? Did you use for the database? What was the tool?  **Student:** SQL.  **Student:** Okay.  **Mohammad:** Did you use anything else? MySQL, Postgre?  **Student:** That one is all.  **Mohammad:** MySQL or PostgreSQL. Both of them. How about SQLite3? Is it a new word? OK.

Mohammad explains the relevance of these tools for the course.

 **Mohammad:** Because we will learn. If you can use one of them, then it's easy. If you can develop a database using one tool, it should be easy for others. Just the setting is a little bit different. Running, for example, some of them, if it's not built in in Python, then you need to run a server and then connect it with that. If it's already built in, this connection as a backend is already available, just you can call the database and use it.  **Student:** It was optional.  **Mohammad:** Yes, it's optional. The most important things, I want to make sure all of you, at least you can use one of these tools. And then for the development, especially for the final assessment, it's optional. You can use any of these tools.

Mohammad continues with the attendance.

 **Mohammad:** Thank you. Next one is Sue. The first name is Penguin. What's your first name? Penguin. Okay. Any questions? Roshan? Any questions? Ravendu?  **Student:** Here.  **Mohammad:** Robindu Malshika Fernando. No? Okay. Another Robindu. Oh, okay. I'm going to...

2. FULL CHINESE TRANSLATION (中文全文翻译)

Mohammad 继续点名，并询问学生们对数据库工具的熟悉程度。

他询问班级是否跟上了进度。

 **Mohammad:** 我们继续下一个。不是 Miss Curry。我们在这里吗?  **学生:** 是的, 老师。  **Mohammad:** 我们在哪里? 好的。有问题吗?  **学生:** 不, 抱歉。一切都好。

 **Mohammad**: 很好。谢谢。

然后他继续点名。

 **Mohammad**: Mahima?  学生: 到, 老师。  **Mohammad**: 都好吗?  学生: 是的。  **Mohammad**: 那 abu 呢?  学生: 到。  **Mohammad**: 有问题吗?  学生: 没有。  **Mohammad**: Ming-Doh?  学生: 到。  **Mohammad**: 怎么样? 生活如何? 有问题吗? 是坚决的“没有”, 还是.....  学生: Mingu? 不, 不, 我在这里。有问题吗? 这对你来说是个新概念吗?  **Mohammad**: 谢谢。

Mohammad 继续点名。

 **Mohammad**: 下一个, 我们在这里。Nazmul Shah。Sheikh Abdul Rashid。Nafseh。Neeraj? Neeraj? Neeraj? 有问题吗? 你熟悉..... Nehima? 我们在这里。Nehima? 有问题吗?

然后他转向讨论关于先前知识和数据库工具的话题。

 **Mohammad**: 在本科阶段, 你们设计过数据库。你们学过这些东西。这算是一种回顾, 复习所有这些内容。你在本科时用什么工具? 你用什么来做数据库? 是什么工具?  学生: SQL。  学生: 好的。  **Mohammad**: 你还用过别的吗? MySQL, Postgre?  学生: 就这些。  **Mohammad**: MySQL 或 PostgreSQL。两者都用过。那 SQLite3 呢? 这是个新词吗? 好的。

Mohammad 解释了这些工具与本课程的相关性。

 **Mohammad**: 因为我们会学到。如果你能用其中一种, 那就很容易。如果你能用一种工具开发数据库, 那么用其他工具也应该很容易。只是设置上有点不同。例如, 运行它们, 有些如果没有内建在 Python 里, 你就需要运行一个服务器, 然后连接它。如果已经内建了, 这个作为后端的连接已经可用, 你只需要调用数据库并使用它。  学生: 那是可选的。  **Mohammad**: 是的, 是可选的。最重要的是, 我想确保你们所有人, 至少能使用这些工具中的一种。然后对于开发, 特别是最终评估, 它是可选的。你可以使用这些工具中的任何一种。

Mohammad 继续点名。

 **Mohammad**: 谢谢。下一位是 Sue。名字是 Penguin。你的名字是什么? Penguin。好的。有问题吗? Roshan? 有问题吗? Ravendu?  学生: 到。  **Mohammad**: Robindu Malshika Fernando。没有吗? 好的。另一个 Robindu。哦, 好的。我要去.....

Level 9 Master of Software Engineering - Professional Software Engineering Course

Attendance & Roll Call (Continued)

STUDENT CHECK-IN

Mohammad continues taking attendance and checking in with students.

 **Mohammad:** Your name is Penguin. What's your first name? Penguin. Okay. Any questions? Roshan? Any questions? Ravendu?  **Student:** Here.  **Mohammad:** Robindu Malshika Fernando. No? Okay. Another Robindu. Oh, okay.

Mohammad proceeds to call on Sana.

 **Mohammad:** I'm going to do Sana. Thank you. Roshan? Where are you, Roshan? Any question? Is it new that designing the database for you, Roshan?  **Student:** No.  **Mohammad:** You knew it before. Sandali? Any question? Sangam?

STUDENT TECHNICAL ISSUE - GIT REPOSITORY

 **Speaker 5:** Yes sir.  **Mohammad:** Any questions?  **Speaker 5:** Not at all but I have some problem in the Git repository.  **Mohammad:** Okay.  **Speaker 5:** I'll try to fix it but.  **Mohammad:** If not please ask. Okay. Are you using the GitHub desktop?  **Speaker 5:** Yes.  **Mohammad:** Or you are using the Git bash?  **Speaker 5:** I use GitHub.  **Mohammad:** No, no, no. What is the tools that you are using to manage the source code? The GitHub desktop?  **Speaker 5:** Yeah, GitHub. Visual Studio.  **Mohammad:** Okay. Through your VS code.  **Speaker 5:** Yes.  **Mohammad:** Are you using the add-ons? You want to manage your code just using the VS code and connected your VS code with the repository. Is there any other tools in the middle to manage your source code or not? Did you install GitHub Desktop?  **Speaker 2:** GitHub Desktop? Yes. You didn't install it?  **Mohammad:** I think first we talked about it.  **Speaker 5:** I have Git Bash. You have Git Bash.  **Mohammad:** You didn't install GitHub Desktop? No, I installed that. You installed that. Okay.

SOURCE CONTROL MANAGEMENT ADVICE

Mohammad provides guidance on managing source code repositories:

- There are **many ways** to control source code in your system, connect it with GitHub, and manage it.
- One method is sufficient for now, but multiple approaches exist.
- Choose the method that is *most comfortable* for you.
- You can *test different ways* and find out which one is easier for you to manage.

 **Mohammad:** If you cannot solve it, please let me know and send the text. All right?

CONTINUED ROLL CALL

Mohammad continues with the attendance process.

 **Mohammad:** After Sangam, the color go sorry any question. Son, yes any question thank you. Any question thank you any question.  **Speaker 3:** I'm sorry. 

Mohammad: No, I'll go.  **Speaker 3:** Yeah, I'll go.  **Mohammad:** Next one is. Any questions?  **Speaker 3:** No.  **Mohammad:** Are you familiar? Did you study the database designing before or is it a new concept? Again.  **Student:** Oh okay, it means it's not new for you.  **Mohammad:** Okay, you learned it before. Good.

FINAL ATTENDANCE CHECK

 **Mohammad:** When you, you here. Absent. No one claims? Shoula? First name?

中文全文翻译

出勤与点名 (续)

学生签到

Mohammad 继续点名并询问学生情况。

 **Mohammad:** 你的名字是 Penguin。你的名字是什么？Penguin。好的。有问题吗？Roshan？有问题吗？Ravendu？ **学生:** 到。 **Mohammad:** Robindu Malshika Fernando。没有吗？好的。另一个 Robindu。哦，好的。

Mohammad 继续点名 Sana。

👤 **Mohammad**: 我要点 Sana。谢谢。Roshan? 你在哪里, Roshan? 有问题吗? 对你来说, 设计数据库是新的内容吗, Roshan? 🗣️ **学生**: 不是。👤 **Mohammad**: 你以前学过。Sandali? 有问题吗? Sangam?

学生技术问题 - Git 仓库

🗣️ **发言人 5**: 到, 老师。👤 **Mohammad**: 有问题吗? 🗣️ **发言人 5**: 没有, 但我在 Git 仓库上遇到了一些问题。👤 **Mohammad**: 好的。🗣️ **发言人 5**: 我会试着修复它, 但是.....👤 **Mohammad**: 如果不行的话请告诉我。好的。你用的是 GitHub Desktop 吗? 🗣️ **发言人 5**: 是的。👤 **Mohammad**: 还是你在用 Git Bash? 🗣️ **发言人 5**: 我用 GitHub。👤 **Mohammad**: 不, 不, 不。你用什么工具来管理源代码? GitHub Desktop? 🗣️ **发言人 5**: 是的, GitHub。Visual Studio。👤 **Mohammad**: 好的。通过你的 VS Code。🗣️ **发言人 5**: 是的。👤 **Mohammad**: 你在用插件吗? 你想只用 VS Code 管理代码, 并将你的 VS Code 连接到仓库。中间有没有其他工具来管理你的源代码? 你安装了 GitHub Desktop 吗? 🗣️ **发言人 2**: GitHub Desktop? 是的。你没安装吗? 👤 **Mohammad**: 我想我们之前讨论过这个。🗣️ **发言人 5**: 我有 Git Bash。你有 Git Bash。👤 **Mohammad**: 你没安装 GitHub Desktop? 不, 我安装了。你安装了。好的。

源代码管理建议

Mohammad 提供了关于管理源代码仓库的指导:

- 有很多种方法可以在你的系统中控制源代码, 将其连接到 GitHub 并进行管理。
- 目前一种方法就足够了, 但存在多种途径。
- 选择对你来说最舒适的方法。
- 你可以尝试不同的方式, 找出哪一种对你来说更容易管理。

👤 **Mohammad**: 如果你解决不了, 请告诉我并发送文本。明白吗?

继续点名

Mohammad 继续点名过程。

👤 **Mohammad**: Sangam 之后, 颜色.....抱歉, 有问题吗? 儿子, 是的, 有问题吗? 谢谢。有问题吗? 谢谢。有问题吗? 🗣️ **发言人 3**: 对不起。👤 **Mohammad**: 不, 我来。🗣️ **发言人 3**: 好的, 我来。👤 **Mohammad**: 下一位。有问题吗? 🗣️ **发言人 3**: 没有。👤 **Mohammad**: 你熟悉吗? 你以前学过数据库设计, 还是这是一个新概念? 再说一遍。🗣️ **学生**: 哦, 好的, 意思是对你来说不是新的。👤 **Mohammad**: 好的, 你以前学过。很好。

最后点名确认

👤 **Mohammad**: 当你.....你在这里。缺席。没人认领吗? Shoula? 名字?

好的, 请查收根据您的要求处理后的 Markdown 文档。

Professional Software Engineering - Session Summary & Logistics

1. Roll Call and Name Pronunciation

👤 **Mohammad**: When you... you here. Absent. No one claims? Shoula? 👤 **Mohammad**: First name? Sofazogod. Sofazogod. Can you pronounce your first name? I want to make sure it's... 👤 **Mohammad**: Shou. Not that far. Thank you. Shou-nim? Thank you. 👤 **Mohammad**: All present and... I think... more any question? 👤 **Mohammad**: Any question? Any question? No. 👤 **Mohammad**: The last one is Z or Shi? True. True. Oh, sorry. Okay. 👤 **Mohammad**: Z is for one J. 🗣️ **Speaker 2**: Yeah. 👤 **Mohammad**: Thank you. J1. 🗣️ **Speaker 2**: Yes. 👤 **Mohammad**: Awesome. Any question? No. Cool. 🗣️ **Speaker 2**: Okay. 👤 **Mohammad**: I try to remember, but maybe it takes a couple of weeks.

2. Course Logistics & Next Week's Plan

2.1. UPCOMING ACTIVITY: DATABASE DESIGN

- **Next Week's Activity**: Mohammad will share an activity requiring students to design a new **Entity-Relationship (ER) diagram** for a new project.
- **Rationale**: Yesterday was a public holiday, so the schedule has been adjusted.

2.2. SCHEDULE ADJUSTMENT FOR SATURDAY SESSION

- **Purpose**: The upcoming Saturday session will be used to cover content that was left behind from this week.
- **Goal**: To design the database and implement the design that has been completed.
- **Outcome**: This will allow the class to move forward with file processing and other topics.

2.3. WORKFLOW AND SUBMISSION

- **Activity Distribution:** The activity will be shared, possibly in one or two parts.
- **Work Period:** Students are expected to work on it during the week.
- **Submission:** Completed work should be shared via a **GitHub** link.

2.4. GENERAL ADVICE AND ENCOURAGEMENT

 **Mohammad:** If you have any questions or comments, please be in touch. Don't leave it be silent. Maybe you're feeling at the beginning for some of you is simple. Please study the share... the link that I shared, the resources that I shared. It will be a little bit further on, it will be really complex for all of you. That's why we're a little bit ready for complex development.

- **Proactive Communication:** Students are strongly encouraged to ask questions and not remain silent if they are struggling.
- **Resource Utilization:** Students should study the shared links and resources.
- **Future Complexity:** The course will become significantly more complex. The current simple topics are preparation for this.

2.5. CLOSING

 **Mohammad:** See you next week on Saturday.  **Speaker 2:** Enjoy your break. Thank you.

Summary & Resources

SUMMARY

Administrative Notifications

1. **Roll Call:** A final roll call was conducted, including clarifying the pronunciation of a student's name.
2. **Next Week's Activity:** A new activity involving **ER diagram design** will be shared.
3. **Schedule Adjustment:** The next Saturday session will be used to catch up on this week's delayed content (database design and implementation).
4. **Submission:** Work must be submitted via a **GitHub** link.
5. **Communication:** Students are urged to ask questions and study the provided resources, as the course will become more challenging.

Core Academic Knowledge Points

1. **Entity-Relationship (ER) Diagram:** The core concept for next week's activity. This is a fundamental data modeling technique used in database design to represent entities and their relationships.
2. **Database Design & Implementation:** The immediate goal is to design a database (using an ER diagram) and then implement that design.
3. **File Processing:** A future topic that will be covered after the database design and implementation phase is complete.

RECOMMENDED LEARNING RESOURCES (LEVEL 9 STANDARD)

To prepare for the upcoming database design activity and the increasing complexity of the course, the following resources are recommended:

1. Textbook:

- **"Database System Concepts" by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan.** This is a definitive, graduate-level text covering ER modeling, relational design, and SQL in depth. Focus on chapters related to **Entity-Relationship Modeling** and **Relational Database Design**.

2. Academic Papers & Standards:

- **Peter Chen's seminal paper:** "The Entity-Relationship Model—Toward a Unified View of Data" (1976). This is the foundational paper for ER modeling and is essential reading for a deep understanding of the subject.
- **ISO/IEC 19761:2011 (COSMIC) or ISO/IEC 14143 (Functional Size Measurement):** While not directly about ER diagrams, understanding functional size measurement is crucial for estimating the complexity of the database and software you are designing, a key skill at the Master's level.

3. Official Documentation & Tools:

- **Draw.io (diagrams.net):** A free, open-source tool for creating professional ER diagrams. Familiarize yourself with its notation for entities, attributes, and relationships.
- **PostgreSQL or MySQL Documentation:** Review the sections on **Data Definition Language (DDL)** to understand how to translate an ER diagram into actual database tables, constraints, and relationships.

专业软件工程 - 课程总结与事务安排

1. 点名与姓名发音确认

👤 **Mohammad**: 当你.....你在这里。缺席。没人认领吗? Shoula? 👤 **Mohammad**: 名字? Sofazogod。Sofazogod。你能念一下你的名字吗? 我想确认一下..... 👤
Mohammad: Shou。差不多。谢谢。Shou-nim? 谢谢。 👤 **Mohammad**: 所有人都到了.....我想.....还有问题吗? 👤 **Mohammad**: 有问题吗? 有问题吗? 没有。 👤
Mohammad: 最后一个 Z 还是 Shi? 对。对。哦, 抱歉。好的。 👤 **Mohammad**: Z 代表一个 J。 👤 发言人 2: 是的。 👤 **Mohammad**: 谢谢。J1。 👤 发言人 2: 是的。 👤
Mohammad: 太好了。有问题吗? 没有。酷。 👤 发言人 2: 好的。 👤 **Mohammad**: 我试着记住, 但可能得花几周时间。

2. 课程事务与下周计划

2.1. 即将进行的活动: 数据库设计

- **下周活动**: Mohammad 将分享一个活动, 要求学生为一个新项目设计一个新的**实体-关系 (ER)** 图。
- **原因**: 昨天是公共假日, 因此时间表已调整。

2.2. 周六课程的时间调整

- **目的**: 即将到来的周六课程将用于补上本周遗留的内容。
- **目标**: 设计数据库并实现已完成的设计。
- **结果**: 这将使课程能够继续推进到文件处理和其他主题。

2.3. 工作流程与提交

- **活动分发**: 该活动可能会分成一到两个部分分享。
- **工作时间**: 学生应在本周内完成。
- **提交方式**: 完成的工作应通过 **GitHub** 链接分享。

2.4. 一般性建议与鼓励

👤 **Mohammad**: 如果你有任何问题或意见, 请保持联系。不要保持沉默。也许你们有些人一开始觉得很简单。请学习我分享的链接和资源。课程往后会变得非常复杂, 对你们所有人来说都是如此。这就是为什么我们要为复杂的开发做好准备。

- **主动沟通：** 强烈鼓励学生提问，如果遇到困难不要保持沉默。
- **资源利用：** 学生应学习分享的链接和资源。
- **未来复杂性：** 课程将变得显著复杂。当前简单的主题是为这个做准备。

2.5. 结束语

 **Mohammad：** 下周六见。  **发言人 2：** 享受你的休息时间。谢谢。

总结与资源推荐

总结

事务性通知汇总

1. **点名：** 进行了最后一次点名，包括澄清一位学生姓名的发音。
2. **下周活动：** 将分享一个涉及 **ER 图设计** 的新活动。
3. **时间调整：** 下周六的课程将用于补上本周延迟的内容（数据库设计与实现）。
4. **提交方式：** 作业必须通过 **GitHub** 链接提交。
5. **沟通：** 敦促学生提问并学习提供的资源，因为课程将变得更具挑战性。

核心学术知识点提炼

1. **实体-关系 (ER) 图：** 下周活动的核心概念。这是一种在数据库设计中用于表示实体及其关系的基础数据建模技术。
2. **数据库设计与实现：** 近期目标是设计一个数据库（使用 ER 图），然后实现该设计。
3. **文件处理：** 一个未来的主题，将在数据库设计和实现阶段完成后进行。

推荐学习资源（LEVEL 9 标准）

为了准备即将到来的数据库设计活动以及课程日益增加的复杂性，推荐以下资源：

1. **教科书：**
 - 《数据库系统概念》(Abraham Silberschatz, Henry F. Korth, S. Sudarshan 著)。
这是一本权威的研究生级别教材，深入涵盖了 ER 建模、关系型设计和 SQL。重点关注与实体-关系建模和关系型数据库设计相关的章节。
2. **学术论文与标准：**

- **Peter Chen 的开创性论文：**《实体-关系模型——迈向数据的统一视图》（1976年）。这是 ER 建模的基础性论文，对于深入理解该主题是必读材料。
- **ISO/IEC 19761:2011 (COSMIC) 或 ISO/IEC 14143 (功能规模测量)：** 虽然不直接关于 ER 图，但理解功能规模测量对于估算你正在设计的数据库和软件的复杂性至关重要，这是硕士级别的关键技能。

3. 官方文档与工具：

- **Draw.io (diagrams.net)：** 一个免费、开源的用于创建专业 ER 图的工具。熟悉其用于实体、属性和关系的符号表示法。
- **PostgreSQL 或 MySQL 文档：** 复习数据定义语言（DDL）部分，以理解如何将 ER 图转换为实际的数据库表、约束和关系。